

**VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF ELECTRONICS**

-----o0o-----



SENIOR DESIGN PROJECT

Implement Tetris on FPGA

GVHD: Trần Hoàng Linh

SVTH: Huỳnh Lê Phi Long

MSSV: 2151111

HO CHI MINH CITY, OCTOBER 2024

MỤC LỤC

Contents

1. INTRODUCTION	5
1.1 Overview	5
1.2 Assigned Duties.....	6
2. THEORIES	7
2.1 VGA Protocol on DE2-115	7
2.2 Audio CODEC.....	11
2.3 PS/2 Keyboard protocol.....	12
2.4 Block Pixel Color Extraction.....	13
3. DESIGN AND IMPLEMENTATION OF HARDWARE	15
3.1 Design overview	15
3.1.1 Play area	15
3.1.2 Functional Blocks Design.....	16
3.1.3 Tetris Tetrominoes	18
3.1.4 Tetris Rotation.....	20
3.1.5 Tetris Movement.....	21
3.1.6 Tetris Mechanism.....	24
3.1.7 Tetris Score.....	25
3.1.8 Tetris Color	26
3.1.9 Tetris falling speed and level.....	27
3.2 Design implement.....	28
3.2.1 Overview	28
3.2.2 VGA controller.....	29
3.2.3 VGA line buffer.....	35
3.2.4 Pixel Decoder.....	37

3.2.5	Game grid	39
3.2.6	Game FSM	40
3.2.7	Generate random piece	42
3.2.8	Update data.....	43
3.2.9	Check lose condition	44
3.2.10	Check illegal move and move	45
3.2.11	Clear line	47
3.2.12	Audio.....	49
4.	RESULT AND CONCLUSION	50
4.1	Design compile result.....	50
4.2	Future goal.....	53
4.3	Picture result.....	54
5.	REFERENCE.....	56

LIST OF FIGURE

Figure 2.1. VGA port in DE2-115	8
Figure 2.2. Horizontal Scanning Process	9
Figure 2.3. VGA Timing Specification Table	10
Figure 2.4. Connections between FPGA and Audio CODEC	11
Figure 2.5. Connection between FPGA and PS/2.....	12
Figure 2.6. PS/2 Sending data.	13
Figure 2.7. Keycode map.....	13
Figure 2.8. Excel deocde from picture.	14
Figure 3.1. The total play area	15
Figure 3.2. Decoration Block	16
Figure 3.3. Letter Block	17
Figure 3.4. Info/Setting Blocks	17
Figure 3.5. Info/Setting Blocks	17
Figure 3.6. Tetris start screen	18
Figure 3.6. Tetris Tetrominoes.....	18
Figure 3.7. Tetris Spawn.....	19
Figure 3.8. Tetris Rotation.....	20
Figure 3.9. Tetris block move left.....	22
Figure 3.10. Tetris block move right	22
Figure 3.11. Tetris block move down.....	23
Figure 3.12. Tetris Mechanism.....	24
Figure 3.13. Tetris Score	26
Figure 3.14. Tetris Color.....	26
Figure 3.15. Overall design	28
Figure 3.16. VGA controller module	29
Figure 3.17. VGA Active Region	30
Figure 3.18. Flow chart of Horizontal Sync	32

Figure 3.18. Flow chart of Vertical Sync.....	33
Figure 3.19. PLL IP of Quartus	34
Figure 3.20. Design of the VGA controller	35
Figure 3.21. VGA line buffer module	35
Figure 3.21. VGA line buffer design	36
Figure 3.22. Piece decoder module	37
Figure 3.23. Piece decode design.....	38
Figure 3.24. Game grid module	39
Figure 3.25. Game grid detail.....	39
Figure 3.26. Game FSM.....	40
Figure 3.27. Generate random piece	42
Figure 3.28. Update infomation.....	43
Figure 3.29. Check lose condition	44
Figure 3.30. Movement.....	45
Figure 3.31. Clear line	47
Figure 3.32. Audio	49
Figure 4.1. Compile result	50
Figure 4.2. Another result from other project doing Tetris	52
Figure 4.3. Start menu	54
Figure 4.4. Setup menu	55
Figure 4.5. Play menu.....	55

1. INTRODUCTION

1.1 Overview

Tetris is a legendary blockstacking game that has become an icon in the history of video games, leaving a lasting impression on generations of players. In this project, I have chosen to recreate Tetris on the FPGA DE2 kit, taking advantage of the VGA display resolution of 640×480 @ 60Hz and using the PS/2 keyboard as the primary input device. The main objective is to deliver a nostalgic experience while leveraging modern hardware technology, thereby combining the familiarity of a classic game with the efficiency and parallelism of digital circuit design.

In Tetris, the player must control and arrange tetromino blocks that continuously fall from the top of the screen into a bounded grid. The primary goal is to rotate and move these blocks so that they form complete horizontal lines. Once a line is fully filled, it disappears, awarding points to the player and freeing up space for continued gameplay. Each tetromino has a unique shape (I, J, L, O, S, T, Z), requiring quick decision making and strategic planning. This interplay between reflexes and logical thinking creates the timeless appeal of the game.

The challenge of Tetris lies not only in handling the increasing speed of falling blocks but also in predicting future moves, planning rotations, and managing limited space effectively. Players must continuously adapt their strategies, balancing short-term survival with long-term optimization. The simplicity of the rules contrasts with the depth of gameplay, making Tetris both accessible and intellectually engaging.

This project does more than replicate the original game, it highlights the practical application of FPGA technology. The 640×480 @ 60Hz VGA output ensures smooth and clear graphics, while the PS/2 keyboard provides a convenient and responsive control interface. By implementing Tetris in hardware, the project demonstrates key concepts of digital design such as finite state machines (FSMs), synchronous logic, random number generation for tetromino selection, collision detection, and memory management for the

game grid. Ultimately, it represents a fusion of classic gaming and modern reconfigurable hardware, offering both entertainment and technical insight.

1.2 Assigned Duties

During the development of the project to implement the Tetris game on the FPGA DE2 kit, the work was divided into three main parts to ensure that the design and implementation process was systematic and efficient.

Task 1: Studying the principles of VGA and PS/2 keyboard communication

The first step focused on learning the fundamental concepts and operating principles of the VGA display interface and the PS/2 keyboard interface.

For VGA, it was essential to understand the signal structure, horizontal and vertical synchronization, and the principles of frame generation on the screen. Through this study, I was able to grasp the methods for generating the required synchronization signals to ensure stable and standard image display. This knowledge was directly applied by programming a VGA controller module capable of producing correct timing signals compatible with the DE2 hardware.

For the PS/2 keyboard, I examined how the keyboard transmits data using scan codes, as well as how the controller interprets and processes input signals. This knowledge formed the foundation for designing the keyboard interface module, allowing the player to input commands to move and rotate tetromino blocks during gameplay.

Task 2: Exploring the FPGA DE2 kit

After understanding the theoretical background of the interfaces, I studied the DE2 FPGA kit itself. This involved learning the basic concepts of FPGA technology, such as programmable logic structure, hardware resources, and device configuration capabilities. I then examined the specific components of the DE2 board, including input/output ports, memory, LEDs, VGA display, PS/2 keyboard, and other peripherals.

Familiarity with these resources enabled me to optimize the usage of the FPGA hardware during programming. In addition, I set up the Quartus development environment, ensuring the ability to compile source code, download it to the board, and verify the system's functionality directly on hardware.

Task 3: Designing the game.

The next step was to construct the basic architecture for the Tetris game. The design was divided into several smaller modules, each responsible for a distinct function such as VGA display control, keyboard input handling, game state management, tetromino generation, movement and rotation logic, collision detection, and line clearing operations. After identifying the necessary modules, I designed how they would communicate through signals and data buses. Each module was programmed step by step, tested through simulation to ensure correctness, and debugged when errors occurred. Once verified, the modules were integrated into a complete system and tested on the physical hardware.

2. THEORIES

2.1 VGA Protocol on DE2-115

The VGA (Video Graphics Array) interface is a widely used standard for displaying graphics on monitors. On the DE2 FPGA board from Altera, VGA is supported through a built in connector that allows image signals to be transmitted directly to an external display. This makes it possible to render the Tetris game in real time on a VGA monitor.

To generate VGA output, the system must produce synchronization signals: horizontal sync (HSYNC) and vertical sync (VSYNC). These signals ensure that the image is drawn line by line and frame by frame in a stable manner. Along with the sync pulses, the FPGA also drives the RGB color signals that determine the color of each pixel.

In the Tetris project, VGA is responsible for displaying the game grid, the falling pieces, the scoreboard, and other elements of the game interface. By programming the VGA controller in Quartus, we can control the resolution (e.g., 640×480 at 60 Hz), assign pixel

colors for each cell of the grid, and update the display every clock cycle to reflect the game state.

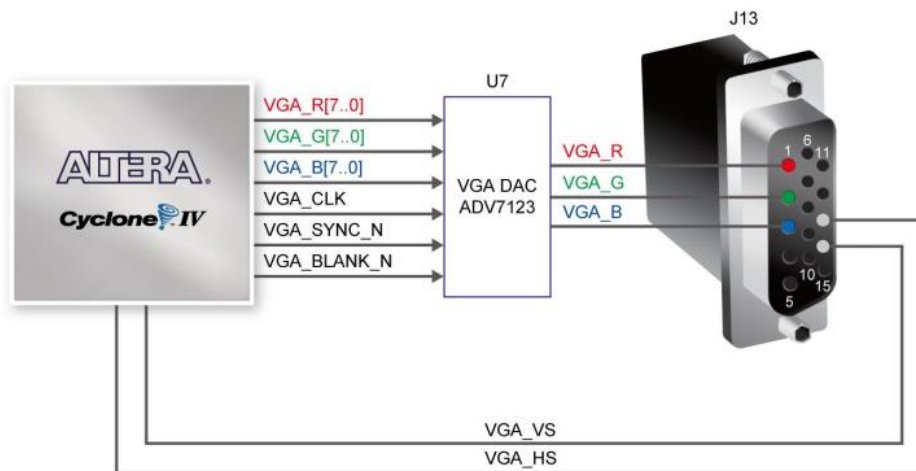


Figure 2.1. VGA port in DE2-115

A VGA display shows images through a scanning process that moves from left to right and from top to bottom. This requires precise coordination between the synchronization signals (HSYNC, VSYNC) and the color signals (RGB). On the DE2 FPGA kit, each color channel (Red, Green, Blue) is represented with 8 bits, allowing for a wide range of brightness levels and producing a rich color spectrum.

Synchronization Signals: HSYNC (Horizontal Sync) and VSYNC (Vertical Sync) are square wave pulses (usually at 3.3V or 5V depending on the hardware). These signals define the exact position of each pixel, ensuring the image is displayed correctly and stably.

Color Signals: Composed of three channels: VGA R, VGA G, VGA B. Each signal ranges from 0V (black) to about 0.7V (maximum brightness). By mixing brightness levels of R, G, and B, different pixel colors are generated up to 2^{24} combinations.

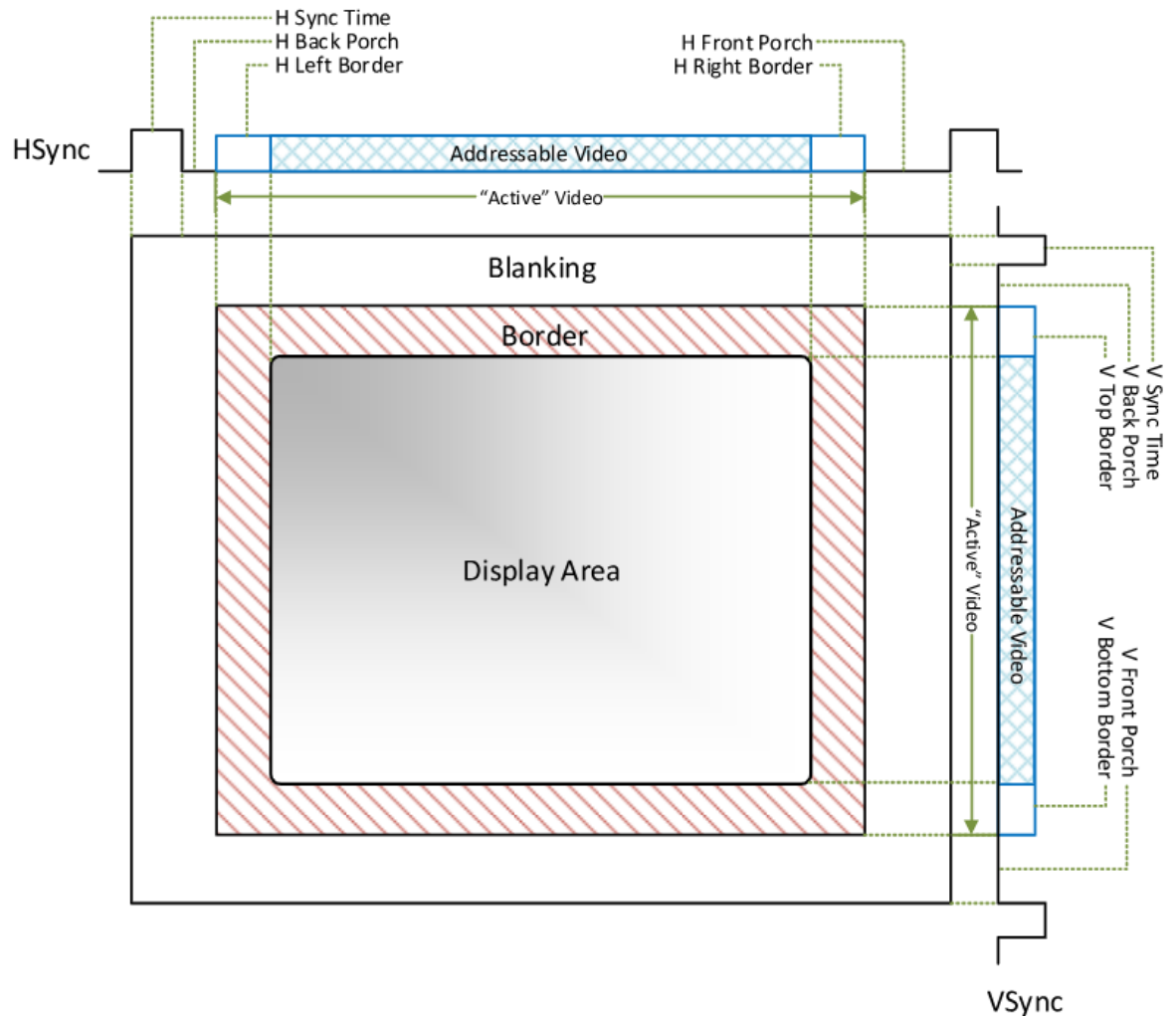


Figure 2.2. Horizontal Scanning Process

Each scan line includes four main phases:

Active Video (Horizontal Display Time): Pixel data (RGB signals) is actively sent to the screen. For example, 640 pixels per line in a 640×480 resolution.

Front Porch: A short pause after finishing the line, before the sync pulse.

Sync Pulse (HSYNC): A synchronization pulse indicating the end of the line and the need to move to the next row.

Back Porch: A short delay before new pixel data starts, allowing the system to stabilize.

s/n	DESCRIPTION	TIME	WIDTH/FREQ
1	Pixel Clock	0.04 μ s	25MHz
2	Refresh rate	16.67ms	60Hz
3	Hor. Sync time	5.12 μ s	128 Pixels/Line
4	Hor. Back porch	0.64 μ s	16 Pixels/Line
5	Hor. Front porch	0.64 μ s	16 Pixels/Line
6	Hor. Display area	25.60 μ s	640 Pixels/Line
6	Total Hor. Pixel	32 μ s	800 Pixels/Line
7	Vert. Sync time	0.064ms	2 Pixels
8	Vert. Back porch	0.928ms	29 Pixels
9	Vert. Front porch	0.320ms	10 Pixels
10	Vert. Display area	15.360ms	480 Pixels
11	Total Vert. Pixel	16.67ms	521 Pixels

Figure 2.3. VGA Timing Specification Table

This table is used to describe how to generate correct VGA signals so that a monitor can display an image at 640×480 resolution, 60 Hz refresh rate.

It provides the exact timing requirements that a VGA controller (implemented in FPGA, ASIC, or graphics card) must follow, including:

Pixel Clock: Defines how fast pixels are sent (25 MHz).

Horizontal Timing (per line): Tells how many pixels are visible (640), and how many are for sync, front porch, and back porch, totaling 800 pixels per line.

Vertical Timing (per frame): Tells how many lines are visible (480), and how many are for sync, front porch, and back porch, totaling 521 lines per frame.

Refresh Rate: Ensures the full screen is redrawn every 16.67 ms (60 Hz).

2.2 Audio CODEC

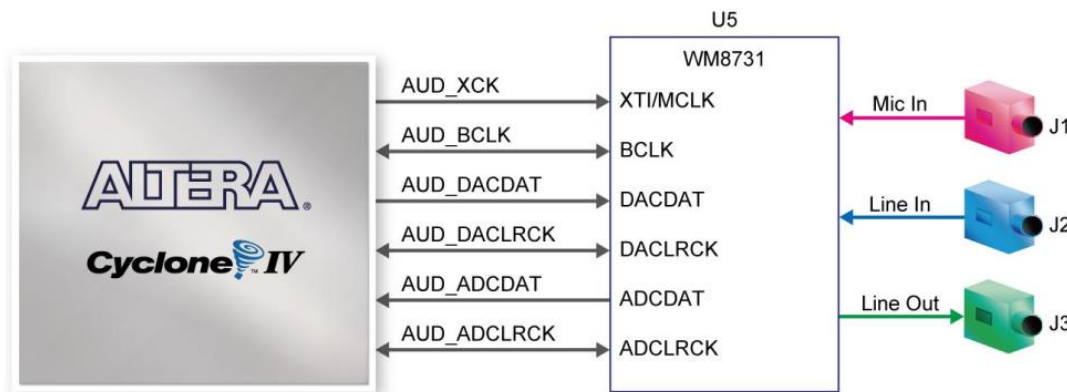


Figure 2.4. Connections between FPGA and Audio CODEC

At a high level, this FPGA project is designed to act as both a controller and a data source for an external audio codec chip. The codec is a mixed-signal device, it converts digital audio data (from the FPGA) into an analog sound signal that can be played through headphones or speakers. The FPGA's role is to generate all the digital signals the codec needs: the control signals, the clocks, and the audio data stream. The codec then uses those signals to operate and produce audio output.

In this design, the FPGA includes a small state machine (inside the `i2c_protocol` module) that automatically sends a series of I²C commands to the codec. Each command writes a value to one of the codec's internal registers, telling it how to behave. The FPGA continuously reads these samples from the ROM, one by one, and serializes them meaning it sends each bit in order onto the `DAC_DATA` line. The rate at which the bits are sent is controlled by `BCLK`, and the channel switching (left/right) is controlled by `DAC_LR_CLK`.

Once the codec receives the audio data and clocks, it converts the digital signal into an analog voltage waveform using its internal digital-to-analog converter (DAC). The analog output is then sent to the codec's headphone or line output pins, where you can connect speakers or headphones. The playback continues as long as `read_enable` is active. If you reset the FPGA, the playback stops, and the entire configuration playback cycle starts over again. This makes it easy to restart or verify operation simply by toggling the reset signal.

2.3 PS/2 Keyboard protocol

The PS/2 interface is a widely used communication standard for connecting input devices such as keyboards and mice to computers or embedded systems. On the Altera DE2 FPGA board, a dedicated PS/2 connector is provided, making it convenient for designers to receive and process keyboard data in their projects.

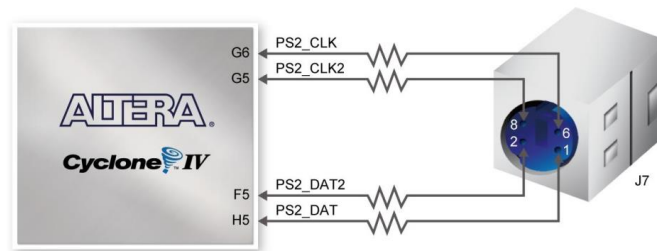


Figure 2.5. Connection between FPGA and PS/2

The PS/2 protocol is a two wire serial communication standard, consisting of:

Clock (PS2_CLK): This clock signal is generated by the PS/2 device (the keyboard). Each clock cycle corresponds to the transmission of one data bit. The falling edge of this signal is used to sample the data line.

Data (PS2_DAT): This line carries the serial data stream. Data is transmitted bit by bit, synchronized with the PS2_CLK signal. The value on PS2_DAT is valid and stable at each falling edge of PS2_CLK.

Each data transfer from the keyboard to the FPGA follows an 11-bit frame structure:

1. **Start bit (1 bit):** Always 0 (logic low), indicating the beginning of transmission.
2. **Data bits (8 bits):** Sent least significant bit (LSB) first. These represent the scan code from the keyboard.
3. **Parity bit (1 bit):** Used for odd parity checking. The total number of logic '1's in the 8 data bits plus the parity bit must be odd.
4. **Stop bit (1 bit):** Always 1 (logic high), marking the end of the frame.

Thus, every key press or release is transmitted in an 11-bit packet.

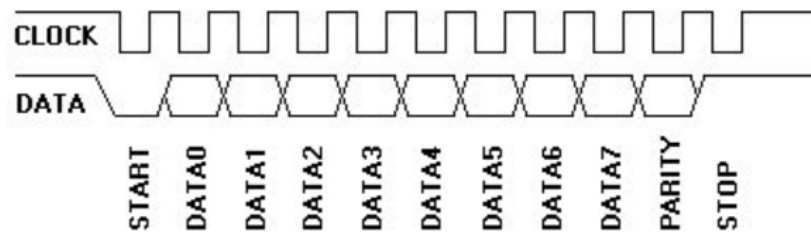


Figure 2.6. PS/2 Sending data.

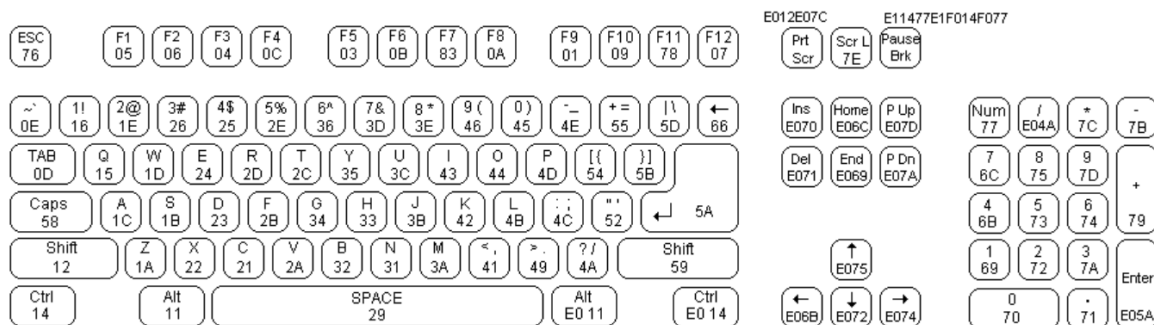


Figure 2.7. Keycode map

The figure is a standard PC keyboard, each key is labeled with its PS/2 scan code. The number inside each key is the hexadecimal scan code sent by the keyboard when that key is pressed.

2.4 Block Pixel Color Extraction

VBA function is designed to extract color channel information from the background of a given Excel cell. Excel represents colors internally as a 24-bit RGB value packed into a Long integer, where each channel (Red, Green, and Blue) is stored in 8 bits. By applying arithmetic operations on this value, the function can isolate and return the desired color component. Such functions are useful for analyzing cell colors, building custom formatting logic, or performing color based calculations directly within Excel.

```
Function GetRed(Cell As Range) As String
    Application.Volatile True
    Dim C As Long, V As Long
    C = Cell.Interior.Color
    V = Int((C Mod 256) / 16)
    GetRed = Hex(V)
End Function

Function GetGreen(Cell As Range) As String
```

```
Application.Volatile True
Dim C As Long, V As Long
C = Cell.Interior.Color
V = Int(((C \ 256) Mod 256) / 16)
GetGreen = Hex(V)
End Function

Function GetBlue(Cell As Range) As String
Application.Volatile True
Dim C As Long, V As Long
C = Cell.Interior.Color
V = Int(((C \ 65536) Mod 256) / 16)
GetBlue = Hex(V)
End Function
```

[illegible]

Figure 2.8. Excel deocode from picture.

The figure illustrates the process of color extraction and storage for a block based graphic representation. On the left hand side, a small image composed of colored cells is presented. Each cell corresponds to a block and is filled with a specific color such as black, red, green, or orange. This visual matrix serves as the raw form of the graphic, resembling how a sprite or game object might appear on a display.

To make this visual information usable for digital systems, each colored block is assigned a unique code. These codes are displayed as hexadecimal values on the right hand side of the figure. By encoding colors in this manner, the visual image is translated into a compact, machine readable table while preserving the layout of the original grid.

The lower part of the figure further refines the representation by separating the color data into its three fundamental channels: red, green, and blue. Each channel is stored as a sequence of hexadecimal characters, corresponding to the intensity of that color component for each block.

3. DESIGN AND IMPLEMENTATION OF HARDWARE

3.1 Design overview

3.1.1 Play area

The play area in this design is organized into a grid consisting of 30 columns and 32 rows, giving a total of 960 individual squares. Each square is defined as a block of 8×8 pixels, which ensures that the structure is simple to manage while still providing enough resolution for clear visual representation. When multiplied out, this configuration corresponds to a display resolution of 256×240 pixels, which aligns well with the requirements of classic game environments such as Tetris.

Although the overall system is implemented on a screen resolution of 640×480 pixels, the actual play area is intentionally made smaller. This decision was not arbitrary but rather a deliberate design choice aimed at improving the system's efficiency. By reducing the effective area, the system is able to process fewer pixels per frame, which in turn helps the hardware run faster and reduces unnecessary computational overhead. This balance between performance and visual clarity is especially important in FPGA based game implementations, where hardware resources are limited and must be used effectively.

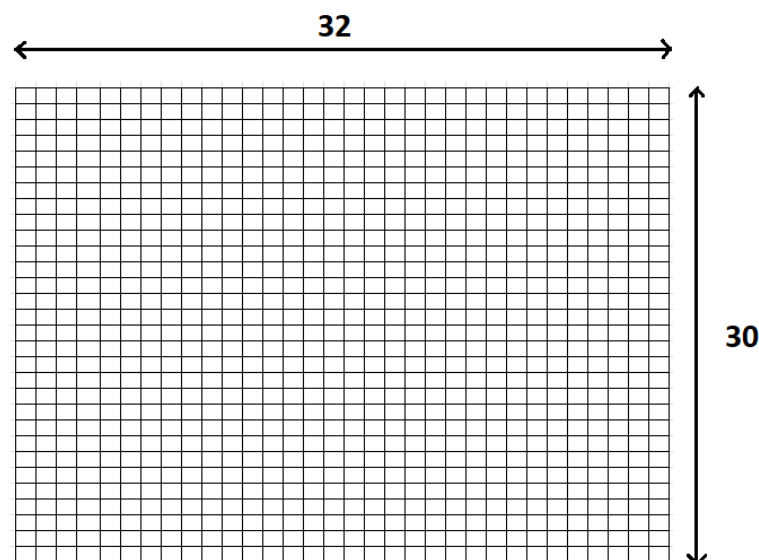


Figure 3.1. The total play area

In addition to the main grid, the play area also integrates an information tab. This tab plays a crucial role in enhancing gameplay by displaying key information to the player. It

typically includes the current game mode or type of gameplay, the score, the number of cleared lines, the current level, and a preview of the next piece that will appear on the grid. Providing this information in real time not only helps the player plan their moves strategically but also replicates the familiar interface of traditional arcade style games.

The design of the play area is a balance between simplicity, performance, and functionality. The choice of a 30×32 grid with 8×8 pixel squares provides a practical compromise between resolution and speed, while the additional info tab ensures that the system delivers a complete and engaging gaming experience.

3.1.2 Functional Blocks Design

In total, the system incorporates more than 200 different block styles that collectively form the Tetris game environment. These blocks are not only responsible for building the core gameplay but also for shaping the visual presentation and user interaction within the system. To manage this variety effectively, the blocks are categorized into several main functional groups, each serving a specific purpose in the overall design.

Decoration Blocks: These blocks are primarily used to enhance the visual appearance of the game. They provide borders, backgrounds, and other stylistic elements that help create a polished and engaging interface, making the play area more visually appealing.

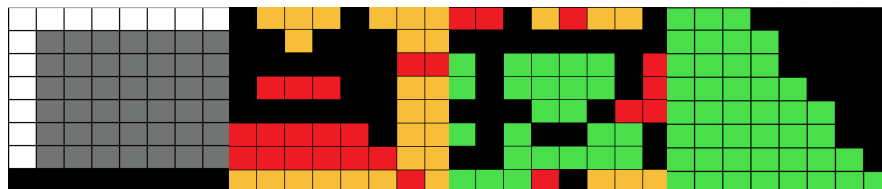


Figure 3.2. Decoration Block

Letter Blocks: This category includes blocks that form letters and symbols, which are used to display text such as game titles, menus, or instructions directly on the screen. By arranging these blocks in patterns, the system can render readable text without the need for complex font rendering engines.

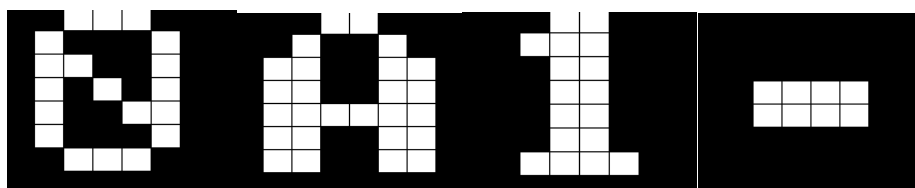


Figure 3.3. Letter Block

Info/Setting Blocks: These blocks are dedicated to displaying essential game information and system settings. They form the structures for showing scores, the current level, the number of cleared lines, and gameplay options. This ensures that the player has continuous feedback on their progress and status throughout the game.

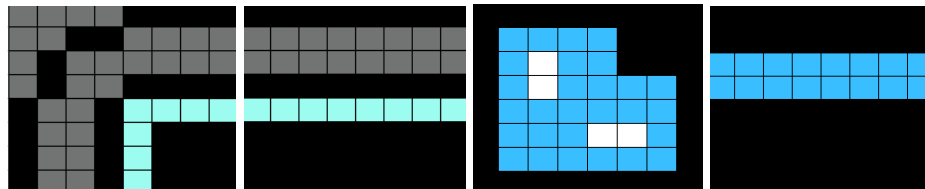


Figure 3.4. Info/Setting Blocks

Play Blocks: These are the core Tetris pieces, often referred to as Tetrominoes. They represent the actual gameplay elements that the player manipulates to clear lines and score points. Their design, rotation, and movement form the heart of the game's mechanics.

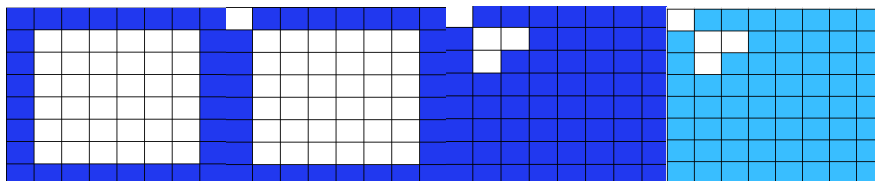


Figure 3.5. Info/Setting Blocks

At first glance, these blocks may appear random or meaningless. However, once they are properly mapped and arranged within the grid, they come together to form a cohesive design that brings the game to life. Through this structured organization, the collection of individual blocks transforms into a visually appealing interface, creating an overall experience that is both engaging and aesthetically pleasing.



Figure 3.6. Tetris start screen

3.1.3 Tetris Tetrominoes

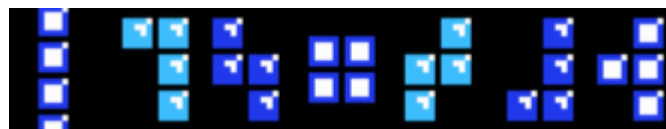


Figure 3.6. Tetris Tetrominoes

The core gameplay of Tetris revolves around a special set of geometric shapes known as Tetrominoes. Each Tetromino is composed of four connected unit blocks, arranged in different patterns. Despite their simplicity, these seven unique shapes create the foundation of the entire game and define its strategic depth.

There are seven standard Tetrominoes, traditionally identified by letters based on their appearance:

I-piece: a straight line of four blocks.

O-piece: a square formed by two by two blocks.

T-piece: three blocks in a row with one additional block centered beneath.

S-piece: a zigzag shape leaning to the right.

Z-piece: a zigzag shape leaning to the left.

J-piece: three blocks in a row with one block attached on the left side.

L-piece: three blocks in a row with one block attached on the right side.

In the hardware design, each Tetromino is assigned a unique **piece ID**, which allows the system to identify, generate, and manipulate the pieces consistently. The blocks that make up each Tetromino are mapped into the **play area grid** by tracking the coordinates of four active unit squares. During gameplay, the piece generator module randomly selects a Tetromino, which is then placed at the top of the grid and begins to descend according to the game's timing logic.

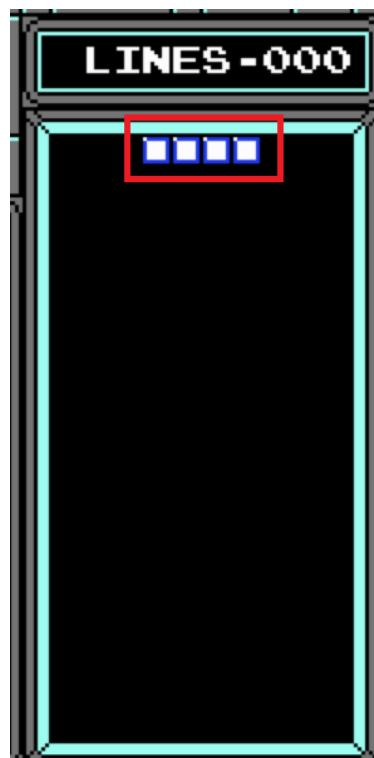


Figure 3.7. Tetris Spawn

3.1.4 Tetris Rotation

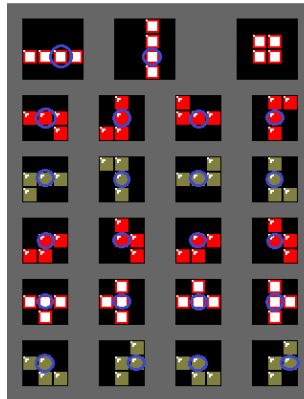


Figure 3.8. Tetris Rotation

One of the most essential mechanics in Tetris is the ability to **rotate Tetrominoes**. Rotation allows the player to change the orientation of a piece so that it fits into different spaces on the playfield. Without this feature, the gameplay would be extremely limited, and many strategic moves would not be possible.

In the hardware implementation, each Tetromino is represented by the coordinates of its four constituent blocks within the play area grid. To perform a rotation, the design uses one of these blocks as the pivot point. The pivot acts as the center of rotation, and the positions of the other three blocks are recalculated relative to this pivot. The standard operation is a 90° clockwise rotation, although the system can also support counterclockwise rotation if needed.

Mathematically, the rotation can be described as a coordinate transformation. Given a pivot at position (x, y) and a block at (x_i, y_i) , the new position after a clockwise rotation is calculated as:

Clockwise

$$X_{\text{new}} = X - (Y_i - Y)$$

$$Y_{\text{new}} = Y - (X_i - X)$$

Anti-Clockwise

$$X_{\text{new}} = X + (Y_i - Y)$$

$$Y_{\text{new}} = Y - (X_i - X)$$

This ensures that the relative distances are preserved and the block is rotated properly within the grid. However, rotation is not always straightforward. Several constraints must be checked before a rotation is allowed:

Boundary conditions: If the new position of any block lies outside the play area (e.g., negative coordinates or beyond the grid width/height), the rotation must be canceled.

Collision conditions: If the new position overlaps with an already occupied block in the grid, the rotation is not permitted.

Special cases: Certain Tetrominoes, such as the O-piece (square), are symmetrical and do not require rotation.

In addition, the design must handle on-the-fly rotation, where the player attempts to rotate a piece while it is already falling. The hardware checks for validity in real time, and only updates the coordinates if the new configuration is legal. This ensures smooth gameplay without glitches or invalid states.

By implementing rotation in this structured manner, the FPGA design faithfully reproduces the behavior of classic Tetris. Rotation not only enhances the gameplay experience but also adds complexity to the hardware design, since it requires dynamic coordinate calculations, collision detection, and boundary checking all within the constraints of real time operation.

3.1.5 Tetris Movement

In addition to rotation, the ability to move Tetrominoes horizontally and vertically is fundamental to Tetris gameplay. Movement allows the player to control where a piece will land, enabling line clears and long-term strategies. In the hardware design, movement is handled by updating the coordinates of all four unit blocks that make up the current Tetromino.

Left Movement: A move to the left is performed by subtracting one unit from the *x*-coordinate of every block. Before the update is committed, the system checks that none of the new positions fall outside the left boundary of the play area or overlap with existing locked blocks. If either condition is detected, the move is canceled.

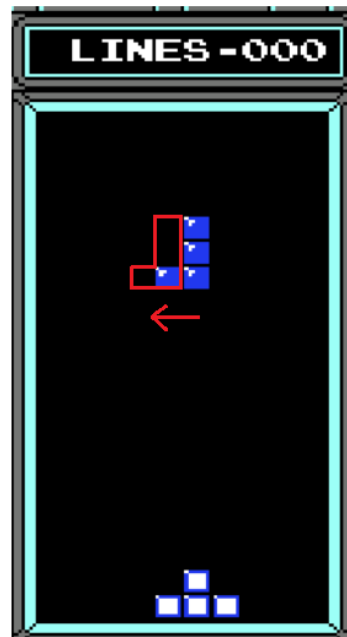


Figure 3.9. Tetris block move left

Right Movement: Similarly, a move to the right adds one unit to each block's x-coordinate. The design applies the same boundary and collision checks, this time focusing on the right edge of the grid. These checks ensure that a piece cannot slide out of the play area or into an already occupied square.

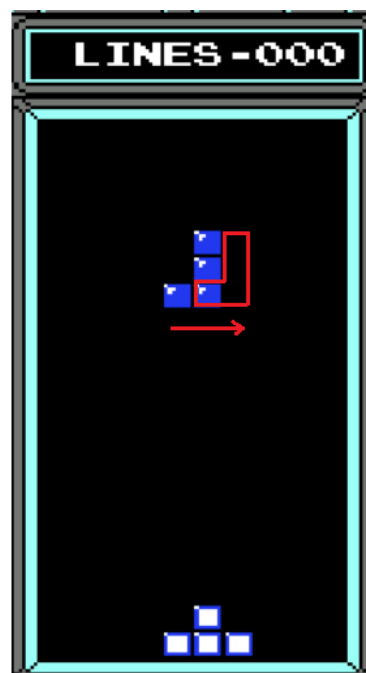


Figure 3.10. Tetris block move right

Downward Movement: The downward motion is the default action, driven by the system's internal clock. At each game tick, one unit is added to the y-coordinate of every block in the Tetromino. This creates the continuous falling behavior of pieces. Players can also accelerate this process by pressing the down key, which forces an immediate update. If a downward move would result in a collision with the bottom of the grid or another locked block, the piece is frozen in place, becoming part of the playfield.

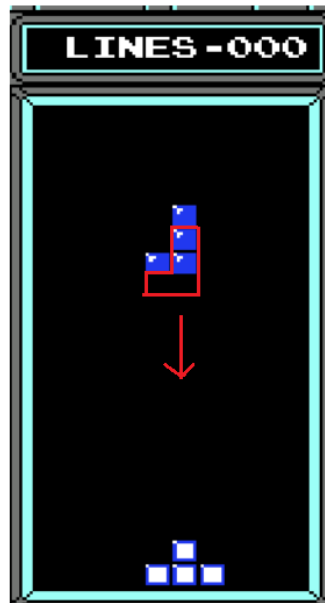


Figure 3.11. Tetris block move down

The hardware implements these movements as a sequence of coordinate updates combined with collision detection. At each step, the new positions are calculated, validated, and then written back into the registers holding the Tetromino's state. This real time process ensures smooth interaction between the player's input (from the PS/2 keyboard) and the graphical output (via the VGA controller).

Together with rotation, the movement logic forms the backbone of Tetris gameplay. Allowing the player to move pieces left, right, and down while continuously checking for validity ensures both an accurate reproduction of the original game mechanics and a robust hardware design that avoids illegal states.

3.1.6 Tetris Mechanism

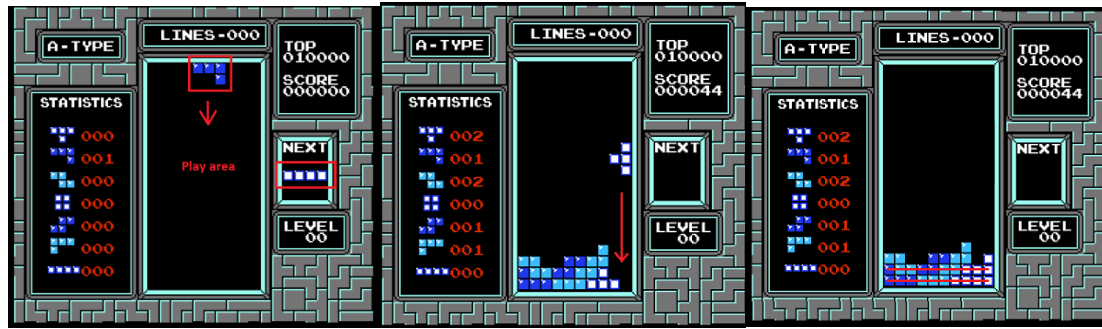


Figure 3.12. Tetris Mechanism

The fundamental mechanism of Tetris is based on a continuous cycle of **falling, locking, and clearing**, which together define the rhythm and challenge of the game. At the start of each cycle, a Tetromino is introduced from the top of the play area. The piece then begins to descend one unit at a time, driven by the game's clock signal. During this falling phase, the player is given control, with the ability to move the piece left, right, or accelerate its downward movement. The player may also rotate the Tetromino to adjust its orientation, allowing it to fit into gaps or form specific patterns within the grid.

The motion of a piece continues until it can no longer move downward. This condition occurs when the Tetromino reaches the bottom boundary of the play area or when it collides with another block that has already been locked into place. At this point, the Tetromino becomes fixed, permanently occupying the cells it has landed on. This locking mechanism ensures that the grid gradually fills with blocks, increasing the level of difficulty as fewer open spaces remain.

Over time, as more pieces are locked into position, horizontal rows begin to form. Whenever a row is completely filled with blocks from one side of the play area to the other, it is recognized as a complete line. The system then performs a clearing operation: the filled line is removed from the grid, and all blocks above it are shifted down by one unit. This process rewards the player with points, creates additional space within the playfield, and preserves the dynamic nature of the game.

The elegance of the Tetris mechanism lies in its simplicity. A repetitive cycle of falling, locking, and clearing creates endless possibilities for gameplay. The player must think strategically, not only about where to place the current Tetromino but also about preparing the

field for future pieces. As the game progresses and the falling speed increases, this mechanism ensures that the challenge intensifies, eventually leading to a game over condition when the stacked blocks reach the top of the play area.

3.1.7 Tetris Score

A central element of Tetris gameplay is its scoring system, which rewards the player based on how many lines are cleared simultaneously. Whenever a horizontal line is filled completely with blocks, it disappears from the play area, and the player earns points. However, the number of points awarded is not the same for every situation. Instead, the game gives increasingly higher rewards for clearing multiple lines at the same time.

Clearing **one line** provides a basic number of points.

Clearing **two lines** together earns a larger bonus, not just double the single line score.

Clearing **three lines** grants even more, further emphasizing efficiency.

Clearing **four lines at once**, using the long “I” Tetromino, is called a **Tetris**. This move gives the highest single action score, significantly more than performing four individual single line clears.

The logic behind this system is to encourage strategic play rather than random placement of pieces. Saving the I-piece and setting up the grid so that four lines can be cleared simultaneously requires foresight, precision, and patience. By providing a high reward, the game motivates players to take risks and plan ahead, which adds both challenge and excitement to the gameplay.

From a design perspective, this scoring mechanic is what makes Tetris more than just a block stacking puzzle. It transforms the game into a balance between short-term survival and long term strategy. Players must constantly decide whether to clear lines immediately to prevent the stack from rising too high, or to wait and set up a potential Tetris for a much greater payoff.

Type	Score
Single	40 x (current level + 1)
Double	100 x (current level + 1)
Triple	300 x (current level + 1)
TETRIS	1200 x (current level + 1)

Figure 3.13. Tetris Score

Player also get some bonus score from the total block that player drop down from the spawn are which maximum at 19 pts.

3.1.8 Tetris Color

In the classic Nintendo versions of Tetris, particularly those released for the NES (Nintendo Entertainment System, 1989) and later popularized throughout the early 1990s, including the 1993 releases, color played an important role in both the Tetrominoes themselves and the evolving background themes.

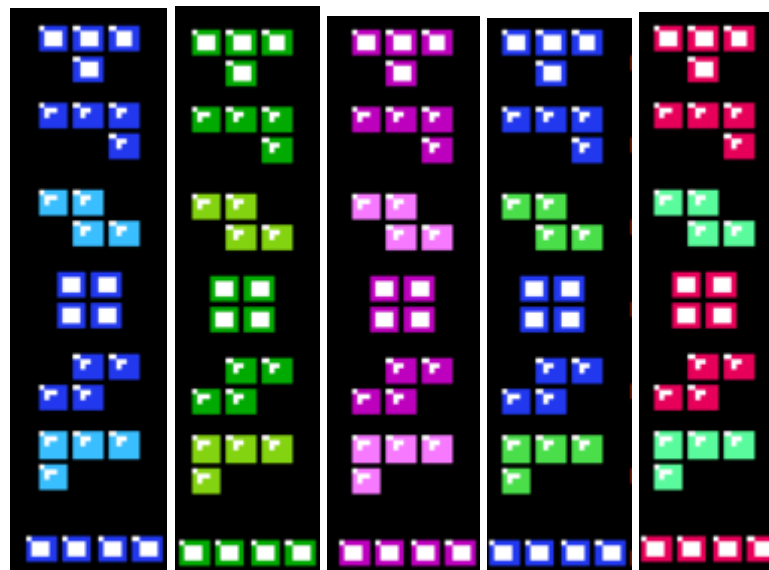


Figure 3.14. Tetris Color

First, the Tetrominoes were given distinct colors to help players differentiate shapes quickly during fast paced gameplay. For example, the long I-piece often appeared as light blue or cyan, the square O-piece as yellow, the T-piece as purple, the S-piece as green, the Z-piece as red, the J-piece as dark blue, and the L-piece as orange. These color assignments became a standard convention that persists in most modern versions of the game.

What made the Nintendo Tetris unique, however, was the way it used changing color themes at different levels. Every time the player advanced in level, not only did the falling speed increase, but the overall background palette of the playfield shifted. For instance, early levels often featured simple black or dark backgrounds with brightly colored pieces, creating a clean

contrast for beginners. As the game progressed, new backgrounds with different color schemes appeared sometimes darker, sometimes to signal the increased intensity of play.

This gradual change of colors served multiple purposes. From a gameplay perspective, it gave the player a sense of progression and achievement as they moved from one level to the next. From a design standpoint, it kept the game visually fresh and engaging despite its simple mechanics. The color changes, combined with the faster drop speed, created an escalating sense of urgency and challenge, ensuring that the game remained exciting for both novice and experienced players.

3.1.9 Tetris falling speed and level

In the game of Tetris, the falling speed of each block, or tetromino, is one of the main factors that determines the difficulty of the game. At the beginning, blocks fall slowly, allowing the player enough time to move and rotate them into position. As the player progresses and clears more lines, the game increases in level, which makes the blocks fall faster and gives the player less time to react. This gradual increase in speed creates a natural and engaging challenge curve for the player.

In an FPGA-based Tetris implementation, the falling speed is controlled by using the system clock. The FPGA typically operates at a high frequency, such as 50 MHz, meaning that the clock produces 50 million pulses every second. By using a counter that counts these pulses, we can measure time intervals and determine when to move the falling block down by one row. The speed formula used in this design is:

$$Speed = \frac{50,000,000}{Current\ level + 1}$$

$$Current\ level = Initial\ level + \frac{Lines\ cleared}{10}$$

This method of adjusting falling speed is both simple and efficient. It uses integer arithmetic, which fits naturally with FPGA hardware design, and it provides a smooth, automatic progression of difficulty. It also closely follows the behavior of the classic Tetris games, where each level roughly doubles the falling speed compared to the previous one. The player can feel a clear difference in pace as they advance, which makes the gameplay more exciting and rewarding.

3.2 Design implement

3.2.1 Overview

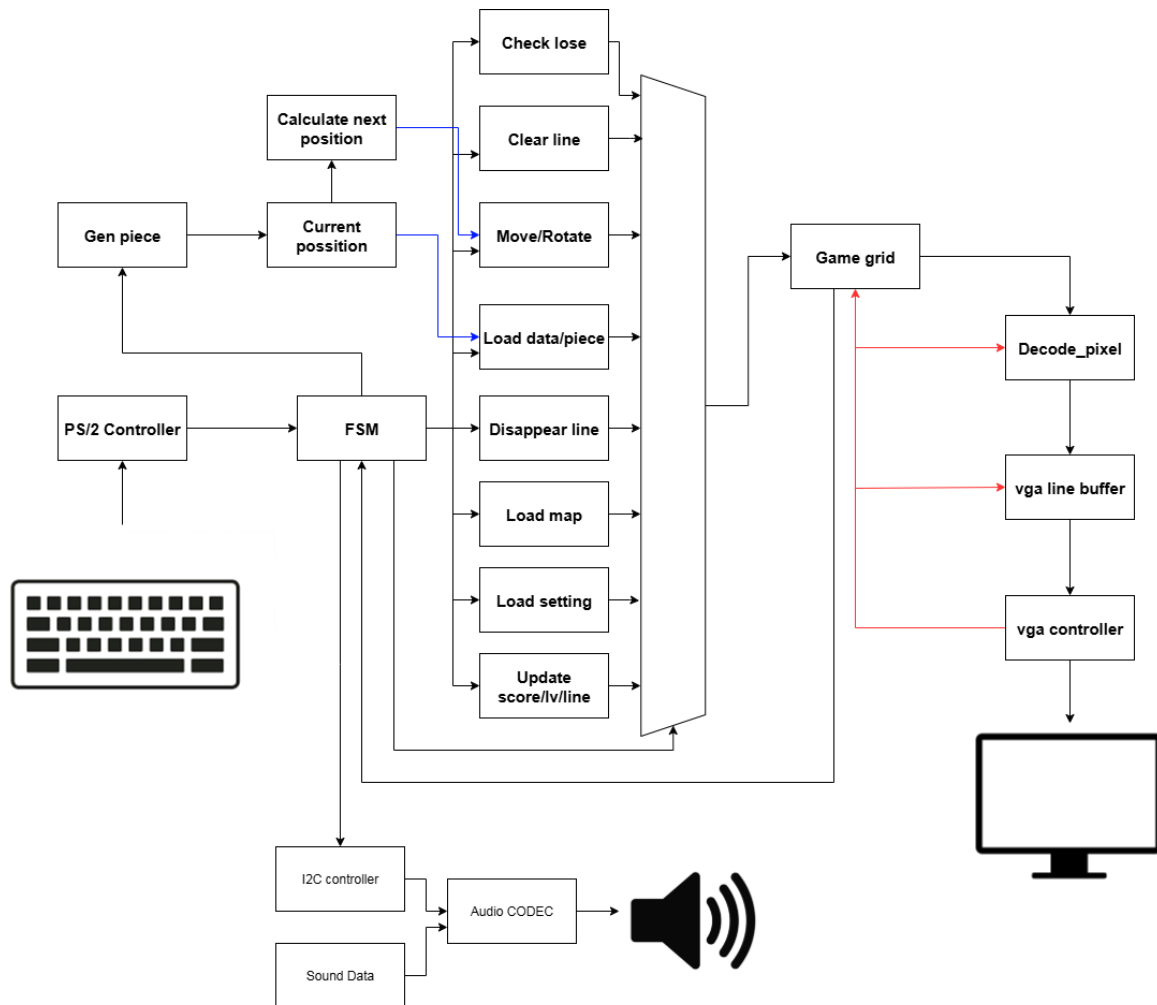


Figure 3.15. Overall design

This diagram illustrates the overall architecture of the FPGA based Tetris system, showing how player input is transformed into visual output on the VGA display. The process begins with the keyboard, connected through a PS/2 controller, which interprets user commands such as moving or rotating a Tetromino. These signals are then processed by the game logic, consisting of modules for piece generation, movement, rotation, collision detection, and line clearing. Together with the finite state machine, this logic maintains the rules of gameplay and updates the state of the playfield.

The playfield itself is represented by the game grid, which stores the position and type of all placed blocks. From this abstract representation, a decode module translates the grid information into pixel level data, assigning the correct RGB color values for each block. This

pixel data is passed through a line buffer, which uses double buffering to ensure smooth, conflict free output. Finally, the VGA controller generates synchronization signals and delivers the pixel stream to the monitor, producing a stable, flicker free visualization of the game.

3.2.2 VGA controller

The VGA controller plays a central role in enabling an FPGA to generate visual output on a standard VGA monitor. Its primary function is to create the timing signals and color outputs required to display images, graphics, or game interfaces on screen. In the context of DE-Series FPGA development boards, such as the DE2-115, the VGA controller acts as the bridge between the digital logic of the FPGA and the analog VGA display standard.

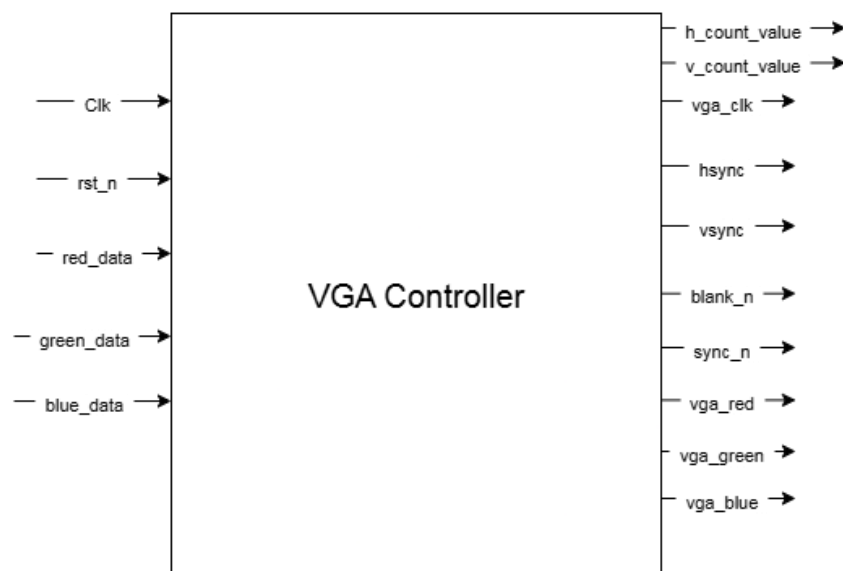


Figure 3.16. VGA controller module

At the heart of this controller is the generation of synchronization signals. VGA displays rely on two key signals: horizontal synchronization (HSYNC) and vertical synchronization (VSYNC). The HSYNC signal defines the beginning of each new line, while the VSYNC signal defines the beginning of each new frame. Together, these pulses ensure

that the pixels generated by the FPGA are placed at the correct location on the screen, maintaining a stable and properly aligned image.

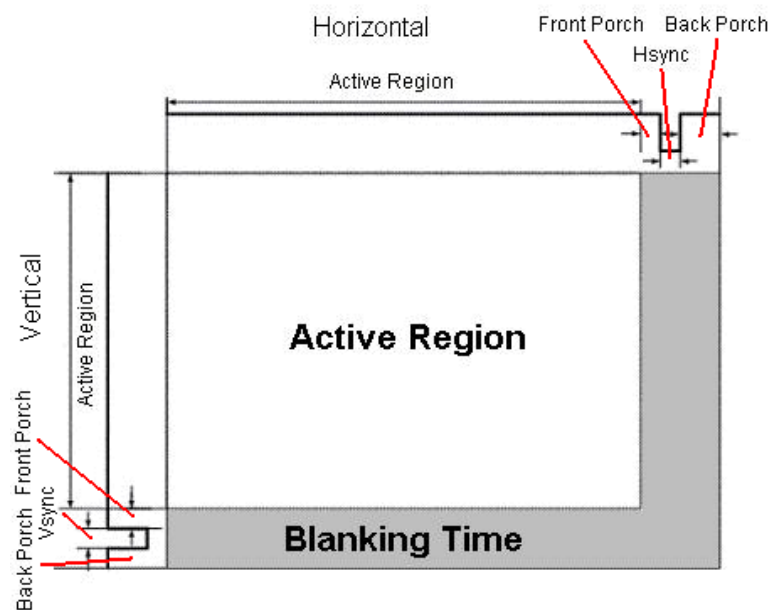


Figure 3.17. VGA Active Region

Active Region (Visible Area): This is the main portion of the frame where actual content is displayed. The coordinate origin (0,0) is located at the top left corner of this region. For the standard VGA 640×480 resolution, the active region consists of 640 pixels horizontally and 480 lines vertically.

Front Porch: The front porch is the short interval that occurs immediately after the visible area but before the synchronization pulse begins. During this time, no image is drawn; it simply provides a buffer period for the display to prepare for the sync signal.

Back Porch: The back porch is the interval that follows the synchronization pulse, before the next visible area begins. Like the front porch, this region does not display content but ensures proper timing margins and stability for the display system.

Horizontal Sync (HSYNC): This is the synchronization pulse that signals the end of one horizontal line and prepares the system to begin scanning the next line.

Vertical Sync (VSYNC): This is the synchronization pulse that marks the end of one entire frame and the start of the next. Total Pixels and Lines: A full VGA frame includes both visible and invisible regions.

For horizontal timing, the total is 800 pixels per line:

640 pixels (visible area)

16 pixels (front porch)

96 pixels (HSYNC pulse)

48 pixels (back porch)

For vertical timing, the total is 525 lines per frame:

480 lines (visible area)

10 lines (front porch)

2 lines (VSYNC pulse)

33 lines (back porch)

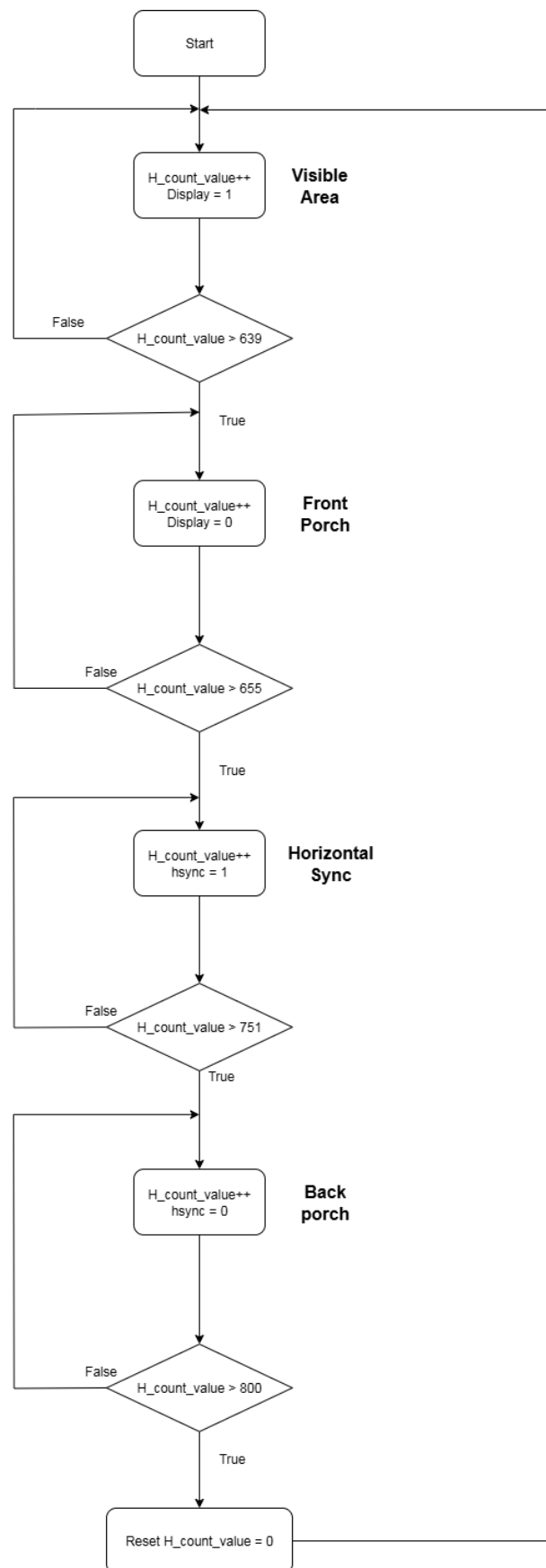


Figure 3.18. Flow chart of Horizontal Sync

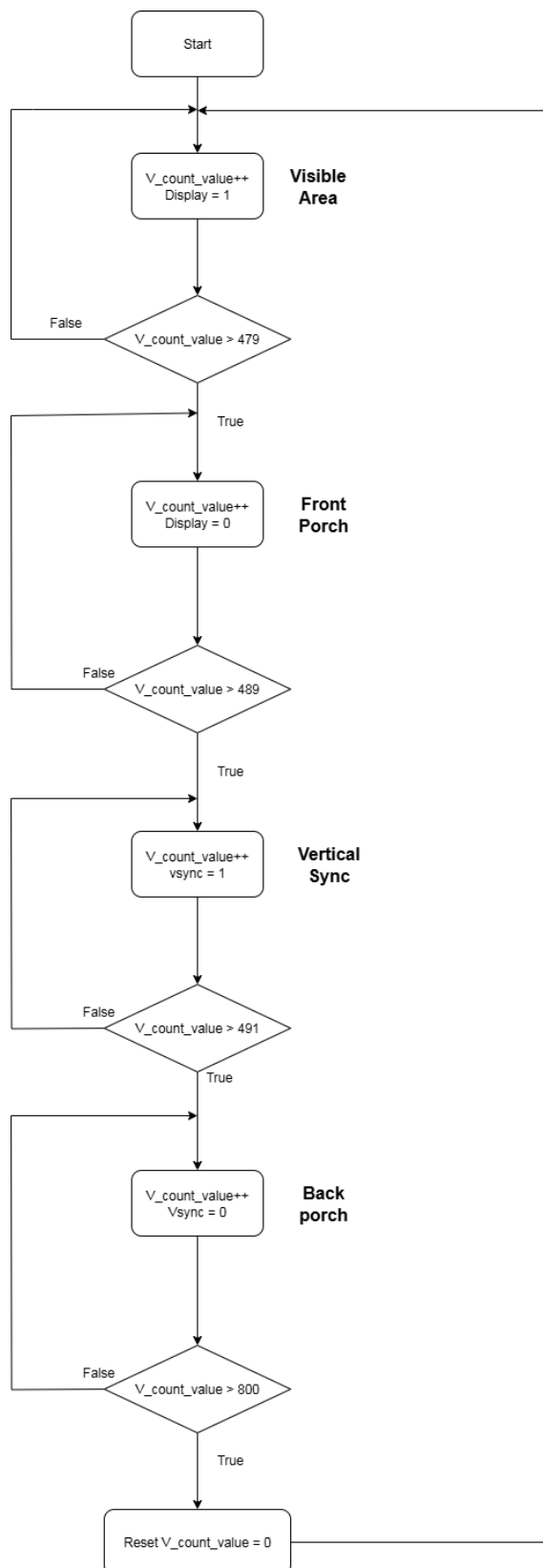


Figure 3.18. Flow chart of Vertical Sync

In addition to synchronization, the controller is also responsible for handling the pixel clock. This clock determines the rate at which pixels are sent to the display, directly influencing the resolution and refresh rate of the output. By carefully managing this clock and synchronizing it with HSYNC and VSYNC, the controller ensures that the monitor receives pixel data at the expected intervals, producing a smooth display without flickering or distortion. For the standard VGA resolution of 640×480 at 60 Hz, this pixel clock must operate at approximately 25.175 MHz. Since the DE2-115 board provides a base system clock of 50 MHz, the design employs a Phase Locked Loop (PLL) to generate the correct 25 MHz clock frequency. By doing so, the VGA controller ensures that the monitor receives pixel data at precisely the expected intervals, producing a smooth display without flickering or distortion.

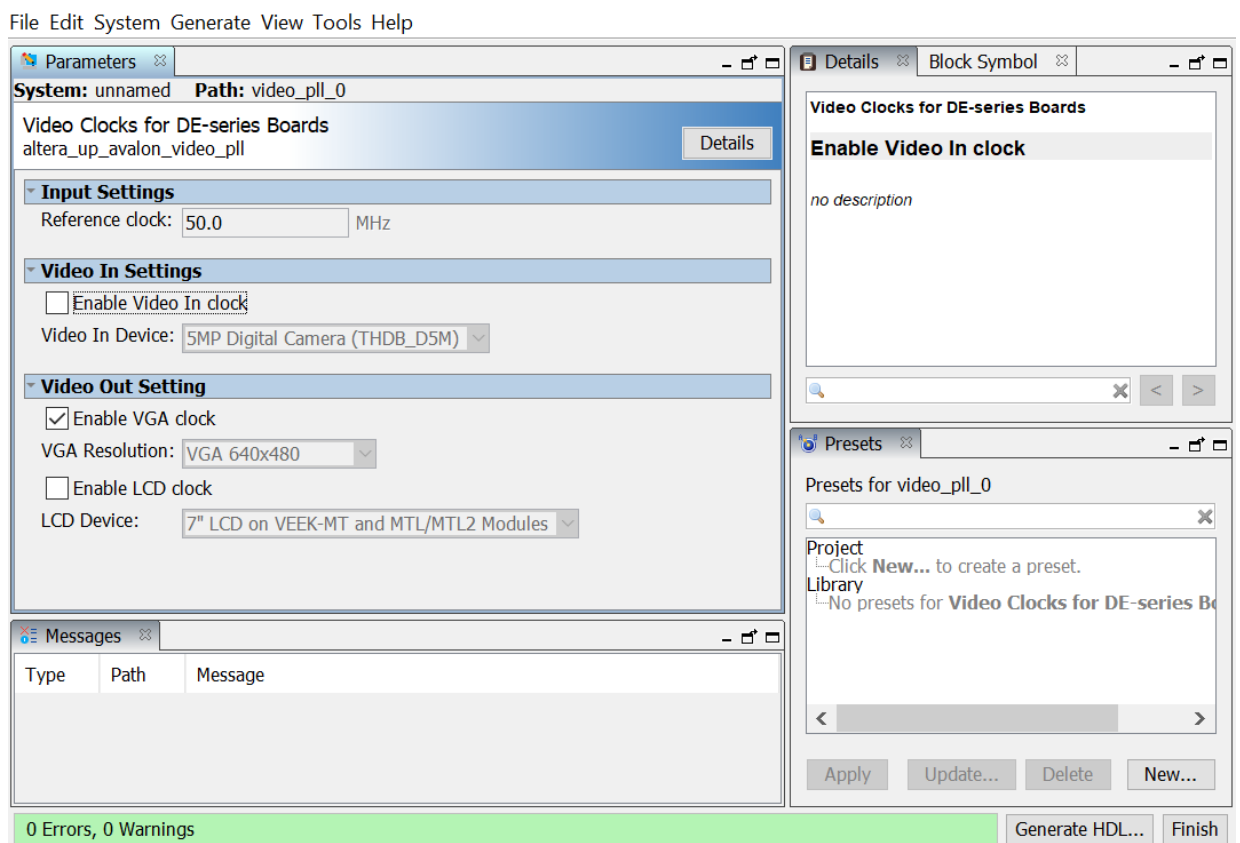


Figure 3.19. PLL IP of Quartus

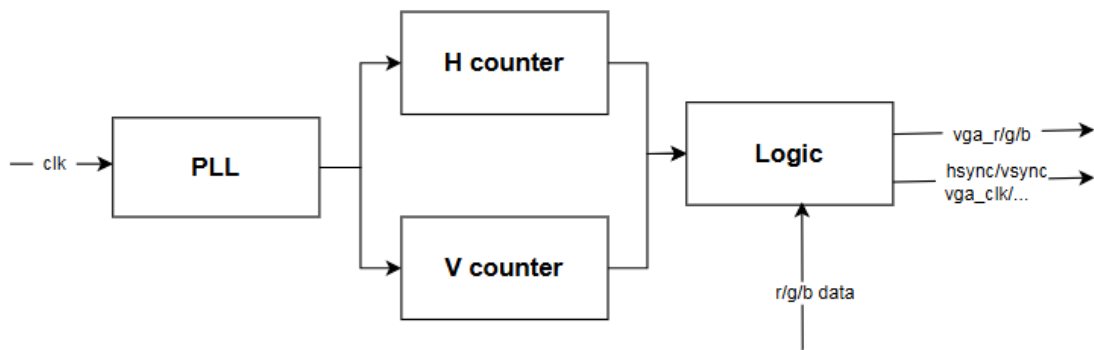


Figure 3.20. Design of the VGA controller

This diagram presents a simplified view of a VGA controller architecture. At its core, the system begins with a PLL (Phase Locked Loop), which converts the FPGA's base clock into the precise pixel clock required for VGA timing. This clock signal drives both the horizontal counter (H counter) and the vertical counter (V counter), which keep track of the current pixel position within a line and the current line within a frame, respectively.

The outputs of these counters are fed into the logic block, which combines timing and color information. This block generates the synchronization signals (HSYNC and VSYNC) that ensure the monitor knows when to start a new line or a new frame. At the same time, the logic also selects the correct RGB data to send as pixel values, producing the visible image on the screen.

3.2.3 VGA line buffer

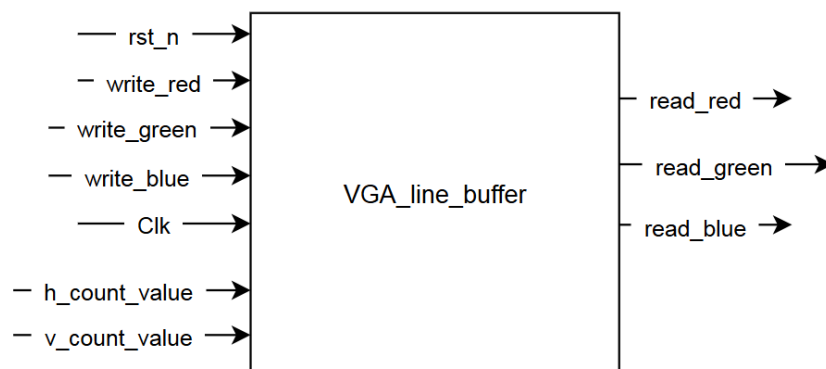


Figure 3.21. VGA line buffer module

The vga_line_buffer module is designed as a double buffered storage system for VGA pixel data, ensuring a continuous and conflict free flow of information to the display. The principle of operation is based on alternating between two independent memory banks, referred to as the odd line buffer and the even line buffer.

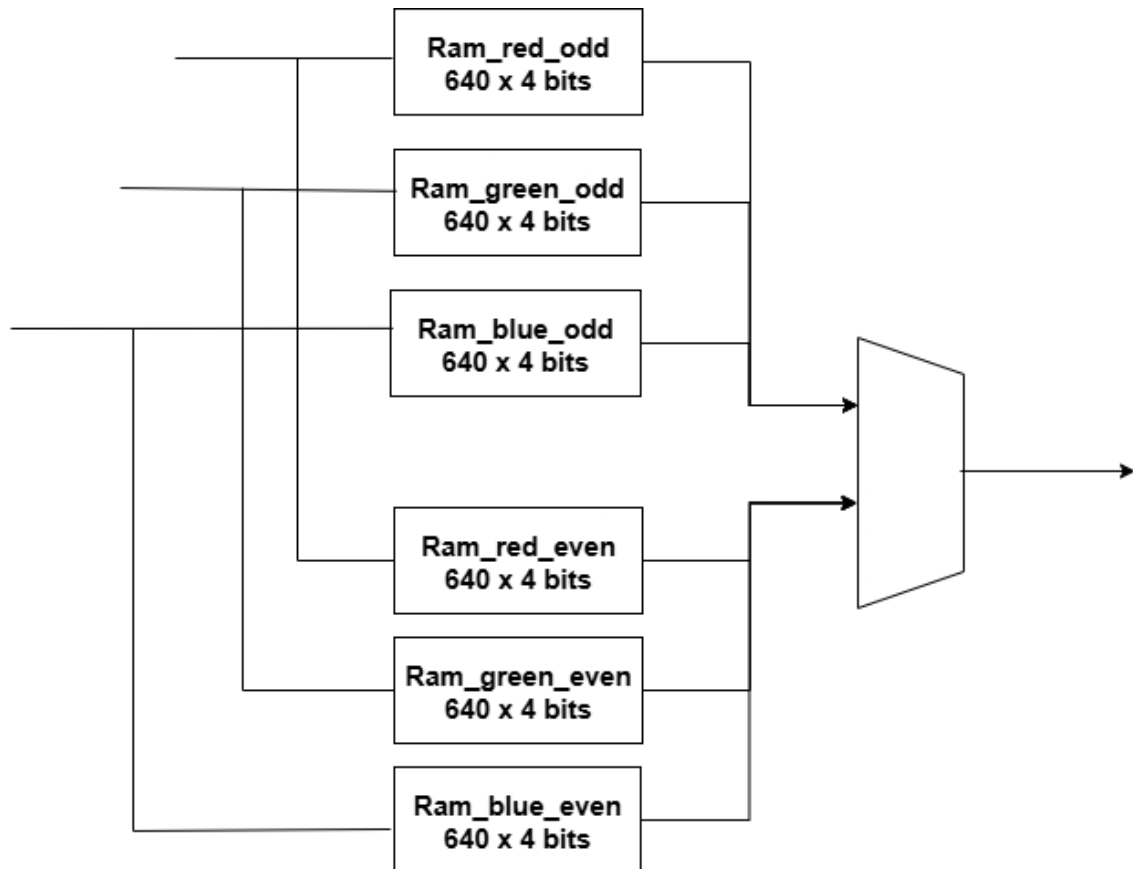


Figure 3.21. VGA line buffer design

During operation, one buffer is dedicated to reading pixel values that are actively being displayed on the VGA monitor, while the other buffer is simultaneously used for writing new pixel data corresponding to the upcoming line. By dividing responsibilities in this way, the system guarantees that the display always receives a stable stream of pixel data without interruption, while new graphics information can be prepared in parallel.

This double buffering strategy is especially important for VGA systems, where strict timing must be maintained at the pixel clock level. Attempting to read and write from the same memory resource simultaneously would risk data corruption or visual artifacts. Instead, by alternating between odd and even line buffers, the vga_line_buffer ensures that reading and writing never interfere with each other.

As a result, the design delivers smooth, flicker free video output, while also allowing new graphics or game logic updates to be seamlessly incorporated into the next line of the frame. This makes the line buffer a critical element in bridging the gap between the game's pixel generation logic and the stringent real time requirements of VGA display output.

The RAM inside the line buffer serves as the fundamental storage element that holds pixel information for each line of the VGA display. In this design, the memory is organized with a data width of 4 bits, which means each location can store a 4-bit value. This size is well suited for representing a single color channel, such as red, green, or blue, allowing fine control of pixel intensity while keeping the memory compact.

Addressing within the RAM is achieved using a 10-bit address bus, providing access to up to 1024 individual memory locations. Although the visible portion of a VGA line requires only 640 pixels, the extended address space ensures alignment and compatibility with the underlying hardware resources of the FPGA. The effective depth of 1024 entries guarantees that each line of video data can be stored, with room to spare for timing overheads or porch intervals.

3.2.4 Pixel Decoder

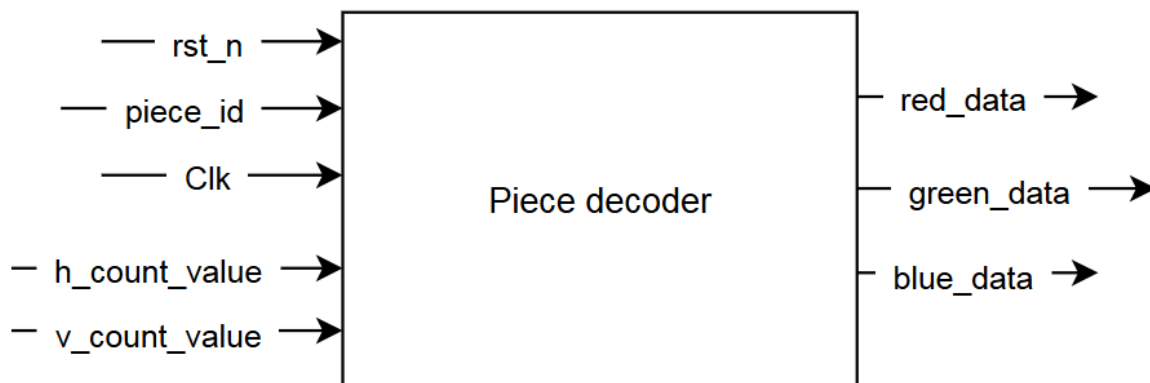


Figure 3.22. Piece decoder module

The piece decoder has the essential mission of translating the game grid representation into actual pixel level data that can be displayed on the VGA monitor. Inside the Tetris system, the game grid only stores high level information, such as which block is present at a certain position and its associated identifier. However, the VGA display requires

detailed information at the pixel scale, with precise values of red, green, and blue for every single pixel on the screen.

This is where the piece decoder comes into play. It receives the identifier of the current piece and the scanning coordinates provided by the VGA horizontal and vertical counters. Using this information, it selects the correct portion of the piece's stored pattern and outputs the corresponding RGB values. In other words, it transforms abstract block data into the fine grained pixel data stream needed by the display.

By performing this decoding process in real time, the piece decoder ensures that each block in the grid is faithfully rendered with the correct shape, color, and alignment. It effectively acts as the bridge between the logical world of the game grid and the visual world of VGA pixels, enabling the Tetris game to be experienced as a smooth, colorful, and fully interactive display on screen.

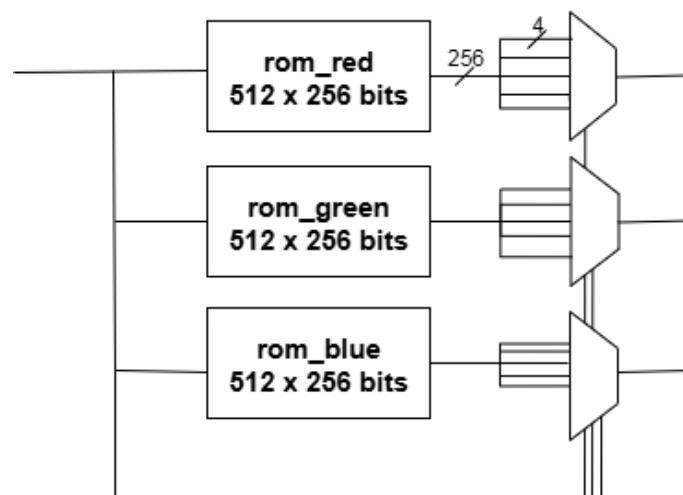


Figure 3.23. Piece decode design

3.2.5 Game grid

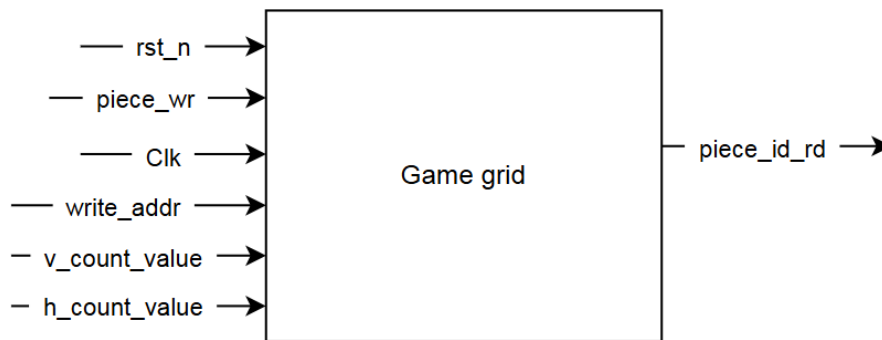


Figure 3.24. Game grid module

At the core of the game grid lies a dedicated RAM block that provides storage for all of the placed Tetris pieces. This memory is organized as 1024 entries, each 9 bits wide, giving it the capacity to represent the entire playfield at the granularity of individual cells. Each 9-bit word encodes the state of a single grid location for example, whether it is empty or occupied, and if occupied, which piece or color it belongs to. With 1024 cells available, the RAM aligns naturally with the dimensions of the Tetris grid, allowing every block on the screen to be mapped directly into memory.

A key feature of this RAM is its ability to support simultaneous reading and writing operations. This means that while the VGA subsystem is continuously reading data to generate the pixel output for the display, the game logic can, at the very same time, write updates into the memory as new pieces fall, rotate, or lines are cleared. This dual access ensures that the display is always up to date with the latest state of the game, without any flicker or delay.

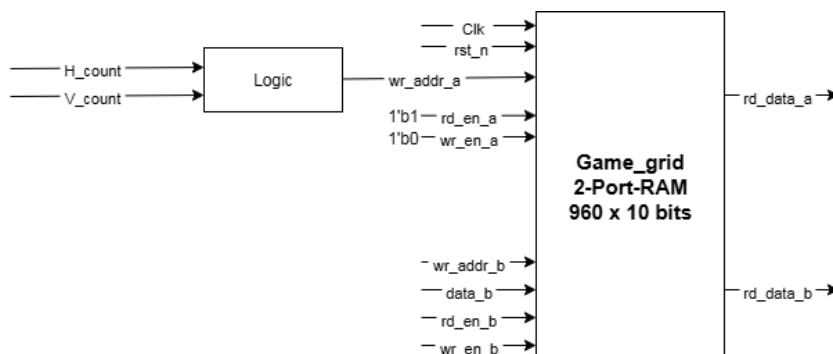


Figure 3.25. Game grid detail

By combining compact storage, efficient organization, and concurrent access, the RAM inside the game grid provides a seamless connection between the logic of Tetris gameplay and the visual rendering on screen. It guarantees that every action taken by the player from moving a piece to clearing a line is reflected instantly on the VGA display, maintaining the smooth and responsive experience that defines the game.

3.2.6 Game FSM

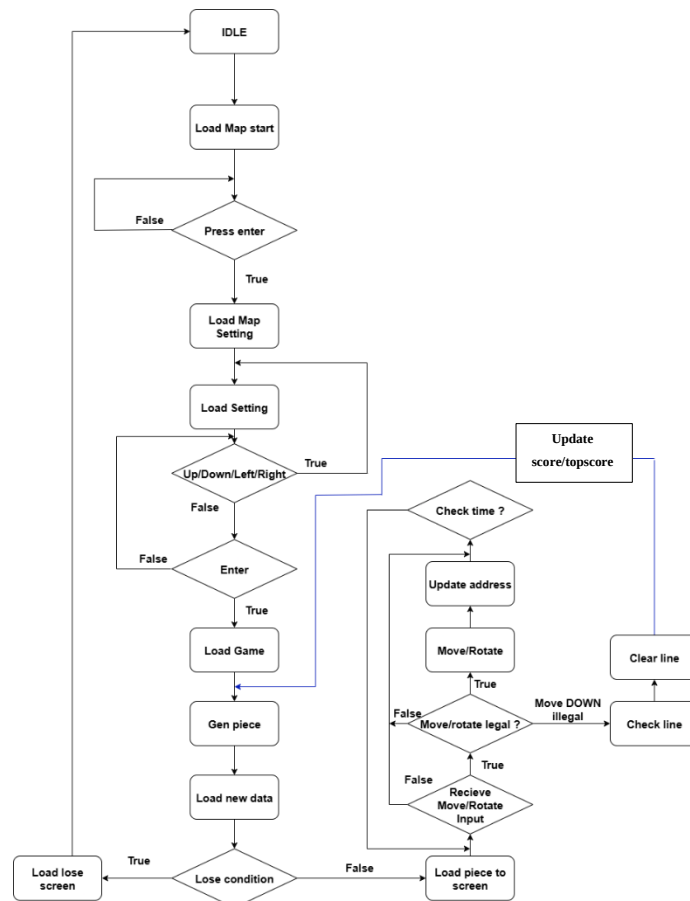


Figure 3.26. Game FSM

The core of Tetris gameplay is managed through a Finite State Machine (FSM), which ensures that all game operations proceed in a structured and predictable manner. On FPGA, where resources are limited and timing is critical, an FSM is particularly well-suited because it allows precise control of events such as drawing graphics, updating memory, and responding to user input.

The FSM begins in the IDLE state, waiting for reset or an initialization signal. Once the system starts, it moves to Load Map Start, which prepares the VGA frame buffer and initializes the playfield grid. This guarantees that the screen displays a clean background before gameplay begins. Next, the FSM progresses through Press Enter, Load Map Setting, and Load Setting states, which give the player the opportunity to configure initial parameters like map style, game speed, or starting level. Navigation through these menus is handled with directional keyboard input (up, down, left, right), while the enter key confirms choices. These states illustrate how the FSM integrates player interaction with hardware display timing in real time.

After configuration, the FSM transitions to the main Load Game state. At this stage, the FSM continuously cycles through the primary game loop. A critical part of this loop is the Check Time state, which determines when the falling block should move down one step. By using a hardware timer synchronized to the 50 MHz system clock, the FSM can implement classic Tetris dynamics, where falling speed increases as levels progress. This ensures a balance between deterministic timing (FPGA clock cycles) and gameplay difficulty progression.

Once the timer condition is met, the FSM executes Update Address to calculate the next positions of the falling piece. Here, player inputs are also considered, enabling lateral movement or rotation. The FSM includes Move/Rotate states that check legality of each action to prevent blocks from passing through walls or overlapping with other placed blocks. These checks reinforce one of Tetris' fundamental mechanics: every move must be physically possible within the grid.

Equally important is the Check Line state, which scans the grid to detect completed rows. If a full row is identified, the FSM enters the Clear Line process. In this stage, memory is updated by shifting all blocks above the cleared row downward, freeing space and rewarding the player with points. This state embodies one of the most iconic features of Tetris: the satisfaction of clearing lines and creating space. The FSM must handle this carefully to ensure no graphical glitches appear on the VGA display while updating the RAM.

Once the line check and clearing are completed, the FSM moves to the Load Piece to Screen state. Here, a new tetromino is generated using a pseudo-random generator (like an

LFSR), placed at the top of the grid, and displayed. This keeps gameplay dynamic and unpredictable, a hallmark of Tetris design.

Finally, the FSM accounts for the Lose Condition. If a new piece cannot be placed because the top rows are already filled, the FSM transitions to the Load Lose Screen state. At this point, the game displays a game-over message, halts further gameplay, and waits for the player to restart. This closure ensures that the FSM maintains a full loop of game logic—from initialization, to continuous gameplay, and finally to endgame handling.

3.2.7 Generate random piece

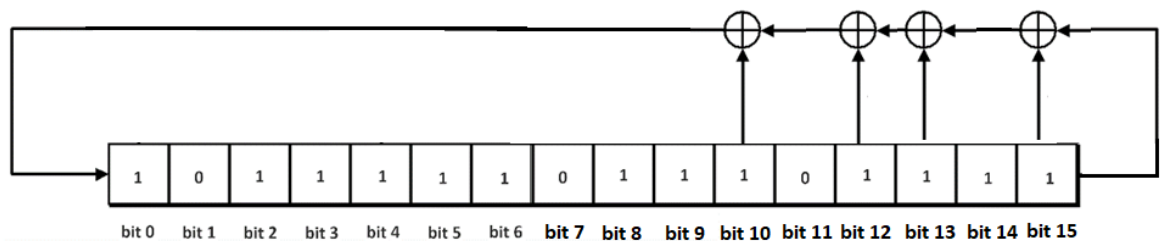


Figure 3.27. Generate random piece

One of the most critical aspects of Tetris is how the next block, or tetromino, is generated. This mechanism defines not only the unpredictability of the game but also its overall fairness and challenge. In implementing Tetris on FPGA, a balance must be struck between simplicity of hardware design and providing a gameplay experience that feels natural and engaging.

In this project, random piece generation is achieved through the use of a Linear Feedback Shift Register (LFSR). An LFSR is a sequential circuit composed of flip-flops and XOR gates that cycles through a long pseudo-random sequence of binary values. Although not truly random, the sequence appears unpredictable to the player, making it a practical and efficient solution for hardware design. Unlike software-based random number generators that may rely on complex algorithms or external entropy sources, the LFSR is lightweight, requires minimal logic, and operates directly with the system clock, making it highly suitable for FPGA implementation.

The design takes advantage of feedback taps at selected bit positions of the register. By XORing these bits together and shifting them into the register, the LFSR ensures that the internal state changes every clock cycle, producing a new pseudo-random value. From this

state, only the lower three bits are extracted to generate the piece identifier. Since three bits can represent numbers from 0 to 7, but Tetris only requires seven distinct pieces, one value is remapped to ensure valid output. This creates a piece ID in the range of 0 to 6, corresponding to the seven tetromino shapes: I, O, T, J, L, S, and Z.

Another strength of this design is the inclusion of a seed-loading mechanism. The seed initializes the LFSR with a specific value, which can be set externally. This feature is highly beneficial in testing and debugging, as it allows the developer to reproduce the exact sequence of pieces across multiple runs. At the same time, changing the seed generates a completely different sequence, giving players varied experiences.

However, it is important to note that this method does not guarantee fairness in distribution. The same piece may appear consecutively, while others may not appear for an extended period. This characteristic reflects early versions of Tetris, where randomness could be both a source of excitement and frustration. In contrast, modern versions often employ the “7-bag” system, where all seven pieces are shuffled and dealt once before reshuffling, ensuring no piece is absent for too long. While more balanced, this approach requires extra logic and memory, which is more costly to implement on FPGA. Thus, the LFSR-based design strikes a good compromise between hardware efficiency and gameplay unpredictability.

3.2.8 Update data

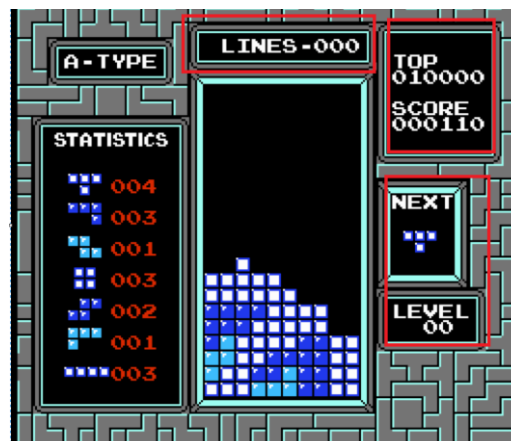


Figure 3.28. Update information

When a falling tetromino finally “locks” in place (because it hit the bottom or landed on top of other blocks), several parts of the game state must be updated together: the board

(grid), the active / next piece, the player's score, the total lines cleared, the current level (and therefore falling speed), and the on-screen displays (next-piece preview, score, top score, level). Doing those updates cleanly and atomically, so the player sees a single coherent change and the hardware logic doesn't glitch, is important in an FPGA implementation. The following essay explains the when, what, and how of those updates and gives practical guidance for implementing them in hardware.

3.2.9 Check lose condition

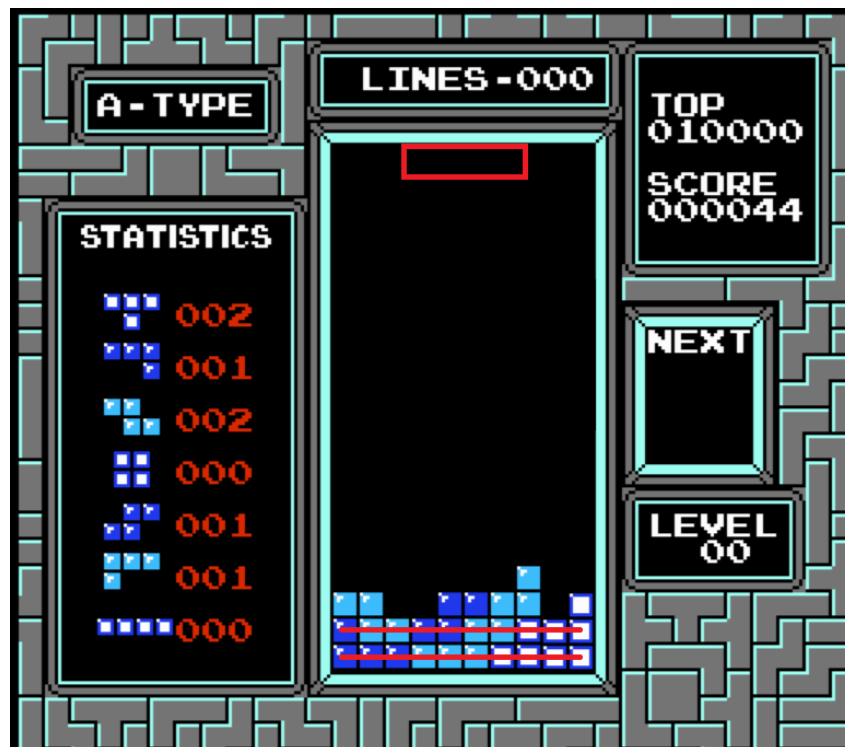


Figure 3.29. Check lose condition

In Tetris the game ends when a newly spawned piece cannot be placed because the target spawn cells are already occupied. This is the canonical and deterministic rule: if the spawn location is blocked, there is no legal move for the new piece and the game is over. Checking for game over is therefore primarily a spawn-collision test: after committing the previous piece and clearing lines, the engine attempts to place the next piece at its default spawn position; if any of the spawn cells are already filled, trigger game-over.

Many simple implementations approximate this by watching a small region near the top of the board commonly the middle 4 cells of the top row and declaring a loss when any of those cells become occupied. That approach often works because most tetromino spawn

positions overlap those top-middle cells. However, it's an approximation and not always correct for all spawn rules or custom spawn coordinates. The robust approach is to test exactly the spawn cells for the specific next piece and its rotation.

3.2.10 Check illegal move and move

In Tetris, the ability of a block to move or rotate is governed by a strict set of collision detection rules, which are at the heart of the game's logic. The first rule concerns horizontal movement to the left. Before a block can shift left, each of its four cells must be checked to ensure that none of them will cross the left boundary of the playfield or overlap with another cell that is already occupied. If any single cell violates this condition, the move is rejected, and the piece remains in its current position. A similar rule applies when moving right. The system verifies that none of the block's cells will exceed the right boundary or collide with existing blocks. Only if the path is clear will the block successfully shift one unit to the right.

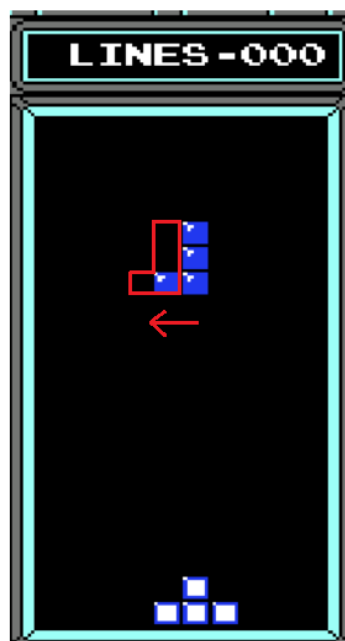


Figure 3.30. Movement

The most fundamental movement in Tetris is downward progression. Gravity forces blocks to fall continuously, but this too is carefully monitored. For each downward step, the system tests whether the cells of the block would extend beyond the bottom of the playfield or land on top of an already placed block. If such a collision is detected, the block is locked into its current location and becomes part of the static playfield. This event is also the trigger

for line-clearing logic, where any fully filled rows are removed to maintain the core mechanic of the game.

Clockwise

$$X_{\text{new}} = X - (Y_i - Y)$$

$$Y_{\text{new}} = Y - (X_i - X)$$

Anti-Clockwise

$$X_{\text{new}} = X + (Y_i - Y)$$

$$Y_{\text{new}} = Y - (X_i - X)$$

Rotation introduces additional complexity, as it requires recalculating the position of each cell relative to a pivot point. When a rotation command is given, the system applies a mathematical transformation—clockwise or counter-clockwise—to generate new coordinates for all four cells. These new positions are then checked against the same constraints as before: they must remain within the boundaries of the playfield and must not overlap with existing blocks. If all conditions are satisfied, the rotation is accepted and the piece updates to its new orientation. If not, the rotation is canceled and the piece retains its previous orientation.

Together, these four checks left, right, down, and rotate form the backbone of Tetris gameplay. They ensure that every action by the player is valid and that the game evolves according to consistent rules. Without these checks, the game would lose both its fairness and its challenge, as blocks could overlap, drift outside the playfield, or behave unpredictably. Thus, the careful implementation of these movement rules is what makes the simple idea of Tetris both engaging and endlessly replayable.

3.2.11 Clear line

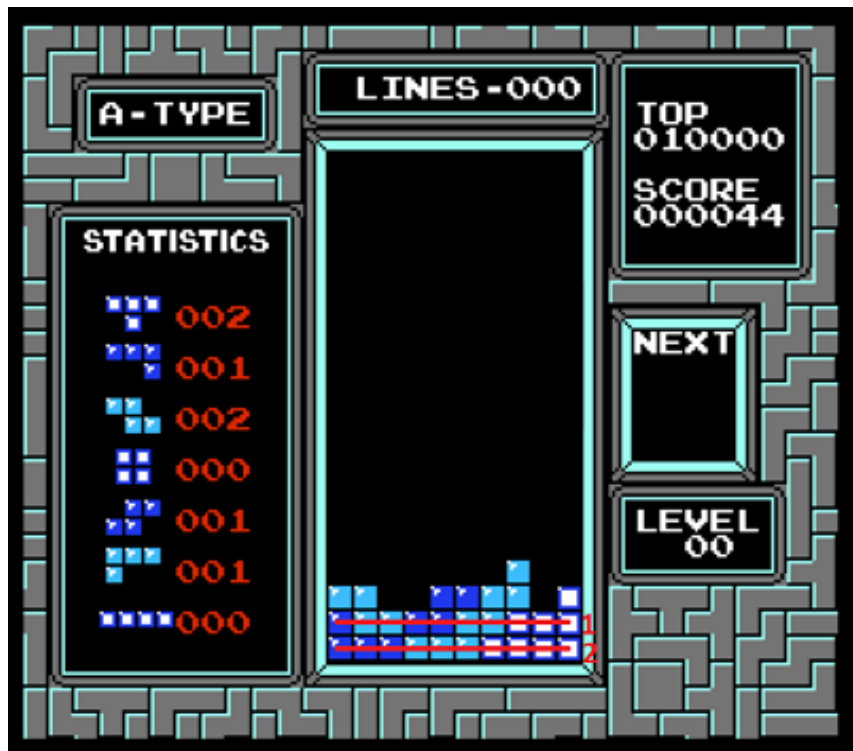


Figure 3.31. Clear line

In Tetris, the act of clearing lines is not only the central mechanic of the game but also the defining feature that gives it challenge and replay value. When a player successfully fills an entire row with blocks, that row disappears, the player earns points, and the game continues at an increasingly faster pace. This simple rule creates tension, strategy, and excitement, which is why Tetris remains one of the most popular games ever created. From a hardware perspective, however, implementing this mechanic is far from trivial. Detecting full rows and deciding how to clear them requires careful design, especially when working with FPGA-based systems where efficiency and resource usage must be optimized. Two main strategies are commonly considered: bottom-to-top line checking and top-to-bottom line checking.

Bottom-to-Top Approach: The bottom-to-top method reflects the natural way we think about Tetris: blocks fall downward, and completed lines are most likely to form at the bottom of the grid. Starting from the lowest row and moving upward allows the system to focus its attention where the action usually happens. This strategy can even optimize performance by ignoring higher rows once enough empty space is detected, which may reduce unnecessary checks.

However, there are significant drawbacks in terms of control complexity. When multiple rows are cleared simultaneously, the rows above must shift down. This shifting can cause new full rows to appear in the same positions, meaning the system may need to re-check the same line repeatedly. For software, this is a manageable loop. But for FPGA-based hardware, where finite state machines (FSMs) drive control flow, this can create complex conditions, additional states, and risks of missed detections if timing is not carefully handled. This makes the bottom-to-top approach less appealing in strict hardware design.

Top-to-Bottom Approach: The top-to-bottom method, on the other hand, may feel counterintuitive at first glance, because fully completed lines are less likely to occur near the top. Yet this approach introduces a stable and predictable sequence that is particularly attractive for hardware implementation. The scan begins at the uppermost row and progresses downward systematically. If a row is full, it is cleared, and all rows above it are shifted downward. Importantly, the algorithm does not need to “rewind” or revisit any previous rows: the scan simply continues with the next line.

This predictable progression simplifies the FSM design dramatically. Instead of having to track dynamic row shifts and re-checks, the FSM can operate line by line with deterministic control logic. This reduces the number of states, avoids potential infinite loops, and ensures that every row is checked exactly once. The result is a more efficient, robust, and easier-to-debug implementation qualities that are highly valuable when working with FPGA hardware.

Impact on Gameplay and System Performance: From the player’s perspective, both methods achieve the same result: lines disappear, rows above shift downward, and gameplay continues. However, from the system’s perspective, the choice of method influences not just performance but also stability. The bottom-to-top method may appear faster for software-based games but risks creating hidden bugs in hardware if not carefully managed. The top-to-bottom method may involve slightly more checking overall but guarantees consistency, correctness, and maintainability in the design.

3.2.12 Audio



Figure 3.32. Audio

Audio is a defining element of Tetris it gives the game its nostalgic feel and provides feedback to the player through music and sound effects. In hardware implementations, the goal is to generate and output those sounds in real time through an audio codec connected to the FPGA. Your setup, with an I²C-controlled codec and 8 kHz, 16-bit audio, is a practical and efficient configuration for low-resource systems such as DE2-115 boards.

The audio codec acts as a bridge between digital logic and the physical sound we hear. Inside the FPGA, all sound data exists as digital numbers, while the codec's job is to turn these numbers into electrical signals that represent audio waves. To make the codec work, it must be configured through I²C communication. During system initialization, the FPGA sends a series of control commands to the codec to set its internal registers. These commands enable the DAC (Digital to Analog Converter), set the sampling rate to 8 kHz, define the audio format as 16-bit data, and control power and volume options. Once configuration is complete, the codec can receive continuous audio data from the FPGA through a separate serial interface, often I²S or a similar protocol.

The choice of 8 kHz sampling rate and 16-bit audio format is both practical and efficient for a hardware-based game like Tetris. At this rate, the system plays 8,000 samples per second, which is more than enough to reproduce the simple melodies and sound effects of classic Tetris. The 16-bit depth provides enough resolution for smooth and clear playback

without noticeable distortion. Using a low sampling rate also reduces the memory size needed to store audio data, allowing the entire music or sound effect to fit comfortably inside the FPGA's internal memory blocks.

4. RESULT AND CONCLUSION

4.1 Design compile result

Flow Status	Successful - Fri Oct 10 06:29:30 2
Quartus Prime Version	21.1.1 Build 850 06/23/2022 SJ I
Revision Name	Tetris_0
Top-level Entity Name	Tetris_0
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	8,322 / 114,480 (7 %)
Total registers	837
Total pins	86 / 529 (16 %)
Total virtual pins	0
Total memory bits	2,838,800 / 3,981,312 (71 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	1 / 4 (25 %)

Figure 4.1. Compile result

The compilation results of the Tetris implementation on the DE2-115 board provide valuable insight into both the efficiency of the design and its scalability for future extensions. The target device used in this project is the Cyclone IV E FPGA (EP4CE115F29C7), a powerful platform equipped with 114,480 logic elements, abundant embedded memory, and extensive I/O resources. The Quartus synthesis and fitting process completed successfully, demonstrating that the design is structurally sound and meets the required timing constraints for reliable operation at the system clock frequency of 50 MHz.

One of the most important indicators is the utilization of logic elements. The design consumes only 8322 logic elements, which accounts for approximately 7% of the total available resources. This remarkably low percentage highlights the simplicity and compactness of the Tetris game logic. The low resource footprint not only confirms that the

design is efficient but also suggests that the FPGA retains ample space for further expansions, such as enhanced graphics, sound modules, or advanced scoring features.

The register usage is equally minimal, with 837 flip-flops employed throughout the design. These registers are primarily used for storing finite state machine (FSM) states, counters, and game logic signals. Given that the FPGA provides nearly fifty thousand flip-flops, the current usage represents only a tiny fraction, further reinforcing the lightweight nature of this implementation.

In terms of I/O utilization, the design uses 86 pins out of the 529 available on the device, which corresponds to about 16%. These pins are primarily dedicated to VGA output, keyboard input, system clock, and reset signals. This usage is modest and leaves a wide margin for additional peripherals or external devices that could enrich the game experience.

A more substantial portion of resources is observed in memory utilization. The design consumes 2,838,800 embedded memory bits, equivalent to 71% of the total 3,981,312 available. This allocation reflects the natural demands of Tetris, where the VGA display, frame buffers, and game grid storage require consistent access to memory resources. Although memory usage is the most significant contributor among all resources, it still remains comfortably within the device's capacity, leaving sufficient headroom for future enhancements such as higher-resolution graphics or more complex animation sequences.

Interestingly, the report shows no usage of embedded multipliers (DSP blocks) and 1 phase-locked loops (PLLs). This is consistent with the nature of the Tetris game, which relies primarily on comparison, control, and memory operations rather than complex arithmetic or frequency synthesis. The absence of DSP and PLL usage further underscores the straightforward and resource-friendly nature of the implementation.

Overall, the results of the compilation indicate a highly efficient design. The FPGA utilization remains well below critical thresholds across all categories, particularly in logic and arithmetic resources. The design is dominated by moderate memory usage, which is expected for a video-based application. These results not only confirm the correctness and stability of the current implementation but also demonstrate the scalability of the project. With such a small fraction of resources consumed, there is considerable potential to extend the system with additional features without risking performance degradation or exceeding device limits.

Device utilization summary:				

Selected Device : 4vsx55ff1148-12				
Number of Slices:	11686	out of	24576	47%
Number of Slice Flip Flops:	1704	out of	49152	3%
Number of 4 input LUTs:	22885	out of	49152	46%
Number of IOs:	38			
Number of bonded IOBs:	36	out of	640	5%
Number of FIFO16/RAMB16s:	129	out of	320	40%
Number used as RAMB16s:	129			
Number of GCLKs:	2	out of	32	6%

Figure 4.2. Another result from other project doing Tetris

The implementation of Tetris on FPGA was compiled on two different devices, namely the Cyclone IV E (Intel/Altera) and the Virtex-5 (Xilinx), in order to evaluate how the same design maps onto different hardware architectures. The compilation results reveal significant contrasts in resource utilization, reflecting not only architectural differences between the two families but also the way synthesis tools optimize the design.

On the Cyclone IV E (EP4CE115F29C7), the project demonstrates extremely modest resource consumption. The design uses only 3,546 out of 114,480 logic elements (3%) and 669 registers, representing a very small portion of the device's sequential resources. Memory utilization is more noticeable, with 500,736 bits used out of 3,981,312 (13%), reflecting the demands of VGA buffering and Tetris grid storage. Nevertheless, this remains within comfortable limits. Importantly, no DSP multipliers or PLLs are consumed, emphasizing the lightweight arithmetic requirements of the design. Overall, the Cyclone IV E compilation demonstrates that the Tetris system is highly efficient and leaves a large margin for scalability.

By contrast, the Virtex-5 (4vsx55ff1148-12) result shows a more substantial portion of the device occupied. The design consumes 11,686 slices out of 24,576 (47%) and 22,885 LUTs out of 49,152 (46%). This indicates that nearly half of the logic fabric is used, a stark difference compared to the Altera device. However, the usage of slice flip-flops is only 1,704 out of 49,152 (3%), suggesting that most of the logic is combinational rather than sequential.

Memory blocks are also heavily utilized, with 129 out of 320 RAMB16s (40%) allocated to the design. I/O usage is modest, with 36 bonded IOBs out of 640 (5%), consistent with requirements for VGA and keyboard connections.

The comparison highlights several key points. First, the same design maps very differently onto Intel and Xilinx devices due to differences in architecture. Altera's Cyclone IV compiles the design into a compact form that uses only a fraction of available logic, whereas Xilinx's Virtex-5 shows much higher utilization, particularly in LUTs and slices. This discrepancy can be attributed to synthesis tool behavior, resource granularity, and optimization strategies. Second, memory is a significant consumer in both cases, reflecting the video output nature of the project, but the Virtex-5 report shows proportionally higher usage (40% compared to 13%).

From a design perspective, the Cyclone IV E offers far more headroom for extensions such as enhanced graphics, audio integration, or more complex scoring systems. The Virtex-5, while still capable, shows a tighter fit and might require more careful optimization if the design were to be expanded further.

In conclusion, compiling the Tetris project on two FPGA families demonstrates both the portability of the design and the architectural contrasts between Intel and Xilinx devices. While the Cyclone IV E provides abundant unused resources and scalability, the Virtex-5 version illustrates higher logic utilization but still maintains feasibility. This comparison underscores the importance of selecting the appropriate FPGA platform based on resource availability, toolchain efficiency, and long-term scalability goals.

4.2 Future goal

While the current implementation of Tetris on FPGA successfully demonstrates the core mechanics of the game piece generation, movement, rotation, collision detection, and line clearing there remains considerable potential for future improvements. These enhancements would not only enrich the gameplay experience but also serve as opportunities to explore more

Another direction involves improving the visual system. The current design outputs graphics through VGA with basic color schemes. Future work could expand the graphics pipeline to include smoother animations, multiple color palettes, and potentially higher screen

resolutions. Leveraging additional memory blocks and hardware acceleration could make the game visually richer while pushing the FPGA's graphics-handling capabilities.

A further extension lies in user interaction. Currently, the game relies on a simple keyboard interface for control. Enhancements could include support for external controllers, joysticks, or even USB interfaces, thereby making the game more versatile and aligning with modern input standards.

Additionally, there is room for advanced gameplay features. These include implementing scoring systems with increasing difficulty levels, multi-player modes where two players compete on the same FPGA board, or special variants such as "Tetris Battle" with unique block behaviors. Exploring these modes would not only improve entertainment value but also highlight how digital logic can manage more complex state machines and parallel processing.

4.3 Picture result



Figure 4.3. Start menu

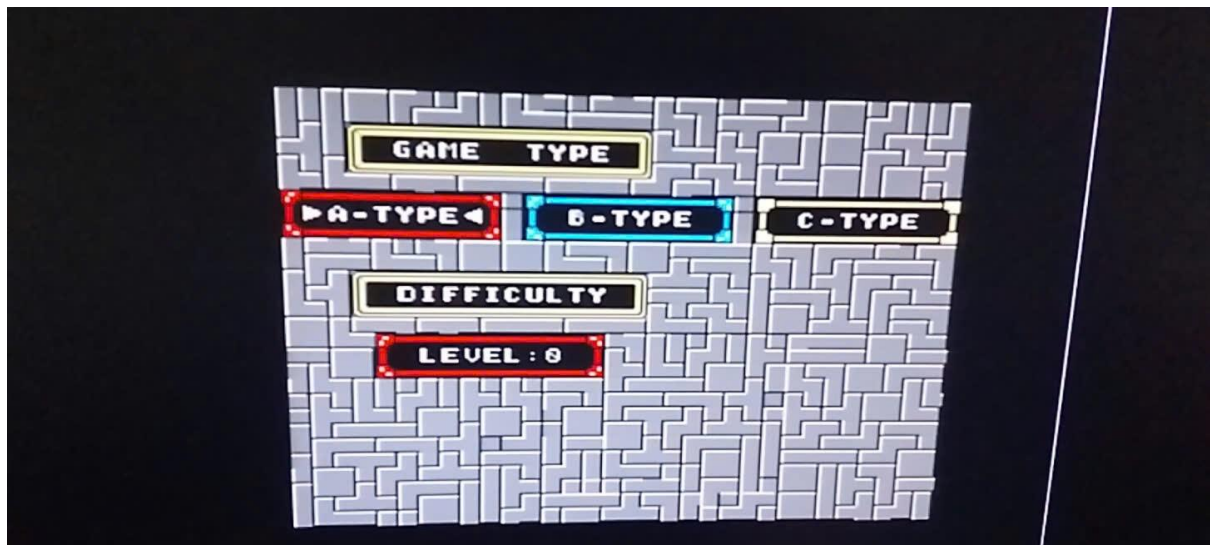


Figure 4.4. Setup menu



Figure 4.5. Play menu

5. REFERENCE

1. Tinylic. (n.d.). *ASIC-FPGA-Tetris*. GitHub repository. Retrieved October 10, 2025, from <https://github.com/tinylic/ASIC-FPGA-tetris>
2. Johan92. (n.d.). *YaFPGA Tetris*. GitHub repository. Retrieved October 10, 2025, from <https://github.com/johan92/yafpgatetris>
3. Kerong2002. (n.d.). *FPGA Tetris*. GitHub repository. Retrieved October 10, 2025, from https://github.com/kerong2002/FPGA_TETRIS
4. Trenz Electronic. (n.d.). *Verilog Tetris*. Trenz Electronic Wiki. Retrieved October 10, 2025, from <https://wiki.trenz-electronic.de/display/PD/Verilog+Tetris>
5. *Nintendo Tetris (NES) – Technical Reference*. (n.d.). *NES Files Documentation*. Retrieved October 10, 2025, from <https://www.nesfiles.com/NES/Tetris/Tetris.pdf>
6. Project FPGA. (n.d.). *VGA Signal and Timing*. Retrieved October 10, 2025, from <https://projectfpga.com/vga/>
7. Valcarce, J. (n.d.). *VGA Signal Format and Timing Specifications*. Retrieved October 10, 2025, from <http://javiervalcarce.eu/html/vga-signal-format-timmming-specs-en.html>
8. Hard Drop Wiki. (n.d.). *Nintendo Rotation System (NRS)*. Retrieved October 10, 2025, from https://harddrop.com/wiki/Nintendo_Rotation_System
9. *Super Rotation System (SRS)*. (n.d.). *Tetris Wiki*. Retrieved October 10, 2025, from https://tetris.fandom.com/wiki/Super_Rotation_System